Fresh Simplex with Height Filter and Timeouts

August 24, 2026

1 Model and definitions

We consider a system of n validators of which at most f are adversarial, where $n \geq 3f + 1$. A *quorum* is any set of at least $2n/3$ validators. Any two quorums overlap in at least $n/3 + 1 > f$ validators, so their intersection contains at least one honest validator.

Definition 1 (Height)**.** *Height* is the finality-gadget counter, separate from slot. The genesis state-height is 1 (with B_{gen} treated as pre-justified and pre-finalized at height 0); height advances by exactly 1 via fixed-target justification (Definition 6) or a timeout (Definition 7).

Definition 2 (Block)**.** A *block* has a *parent* block, a *slot* field, and a set of votes (Definition 4). The *genesis block* B_{gen} has no parent and slot 0. Slots strictly increase along parent links: $B.\,\text{slot} > B.\mathsf{parent}.\,\text{slot}$ for every non-genesis B.

Definition 3 (Chain)**.** A *chain* is a path from B_{gen} to some block B via parent links. Chains are in bijection with their tips, and we identify the two. Blocks form a tree rooted at B_{gen}; we write $B \preceq B'$ when B is an ancestor of B' (or $B = B'$), $B \prec B'$ for strict ancestry, and $B \sim B'$ (*compatible*) when $B \preceq B'$ or $B' \preceq B$. Two chains *conflict* when their tips are not compatible.

Definition 4 (Vote)**.** A *vote* is a signed tuple

$$(\mathsf{validator},\ \mathsf{height},\ \mathsf{target},\ \mathsf{finalize\_height},\ \mathsf{finalize\_target}),$$

where $\mathsf{validator}$ is the signer, height is a height or $\perp$, and $\mathsf{target} \in \{\perp\} \cup \text{Blocks}$. A vote with $\mathsf{target} \neq \perp$ (necessarily carrying a height) is a *justification vote*; a vote with $\mathsf{target} = \perp$ and a height is a *timeout vote*; a vote with an *empty voted checkpoint*, $\mathsf{target} = \perp$ *and* $\mathsf{height} = \perp$ (the empty $\mathsf{Checkpoint}()$, written $\mathsf{Checkpoint}()$), is an *empty vote* (Definition 29), which makes no claim about any height and so contributes to neither a justification (Definition 6) nor a timeout (Definition 7); it acts only through its head field and its finalize commitment. The pair ($\mathsf{finalize\_height}$, $\mathsf{finalize\_target}$) is either both $\perp$ or a commitment to finalize a block at a height. A vote included on a chain must reference a target (and finalize target, when present) that already exists on that chain when non-$\perp$; in particular, a block cannot include a vote that names the block itself, since naming it requires its hash. An FG attestation additionally carries a *head field* $v.\mathsf{head} \in \text{Blocks}$ (the SG latest message of Section 5); the head field is ignored by every state transition of this section and enters fork choice immediately when the attestation is received and validated (Definition 15).

State. The *state after block* B, written $\sigma[B]$, is the tuple

$$(L,\ s,\ h,\ s_h,\ \mathsf{targets},\ \mathsf{timeouts},\ J,\ h_j,\ F,\ P),$$

with components:

- L: the *chain head* ($L = B$ after processing B).

- s: the *current slot*, incremented once per slot by processSlot (Figure 1).
- h: the *current height*.
- s_h: the slot of the block whose processing last advanced h on this chain (0 if no advance has occurred).
- targets$[1..n]$: for each validator i, the target of i's justification vote at the current height ($\perp$ if none processed).
- timeouts$[1..n]$: for each i, a boolean that is `true` iff i's vote at the current height set the timeout marker — either a timeout vote (target $= \perp$ with height $= \sigma.h$) or a height-fresh justification vote on this chain. An empty vote (target $= \perp$, height $= \perp$; Definition 29) sets no marker, since it is not a vote at the current height.
- J, h_j: the most recently justified block and its height.
- F: the most recently finalized block.
- $P \subseteq \{1, \ldots, n\}$: validators who have cast a vote with finalize_height $= h_j$ and finalize_target $= J$.

The genesis state is

$$\sigma_{\text{gen}} \coloneqq (B_{\text{gen}},\ 0,\ 1,\ 0,\ \perp^n,\ \texttt{false}^n,\ B_{\text{gen}},\ 0,\ B_{\text{gen}},\ \emptyset),$$

with $\sigma[B_{\text{gen}}] \coloneqq \sigma_{\text{gen}}$: B_{gen} is treated as pre-justified and pre-finalized at height 0, so the genesis state-height is 1 and $h_j = 0$ denotes the genesis justification. When unambiguous we drop the chain prefix and write L, s, h, etc. for the components of $\sigma[L]$.

Definition 5 (State-height)**.** The *state-height* of a block B is $\sigma[B].h$: the height after processing B on its own chain.

Height freshness. A justification vote v (i.e., $v.\text{target} = T \neq \perp$) is *fresh* on a chain with head state $\sigma[L]$ iff

$$\sigma[L].h = v.\text{height} \ \wedge\ T \prec L \ \wedge\ T.\,\text{slot} \geq \sigma[L].s_h :$$

the chain is at v's height, T is a strict ancestor on this chain, and T's slot is no earlier than where the chain's current height began. A timeout vote ($v.\text{target} = \perp$) is fresh on the chain iff $\sigma[L].h = v.\text{height}$. Intuitively, a fresh justification vote attests that its signer saw T as a chain of height $v.\text{height}$.

Remark 1 (Freshness via T's own chain). For a justification vote with target $T \prec L$, the constraint $T.\,\text{slot} \geq \sigma[L].s_h$ says that T lies in the current state-height interval of L. Equivalently, it means $\sigma[T].h = \sigma[L].h$; this equivalence is proved as Lemma 2.

Freshness ensures that every justification at height h is witnessed by targets whose own chains reached state-height h, and pins both timeout and justification contributions to the chain's current height.

Definition 6 (Fixed-target justification)**.** A block T is *justified at height h* on a chain with state $\sigma[L]$ with $\sigma[L].h = h$ iff

$$\big|\{i \,:\, \text{targets}[i] = T\}\big| \ \geq\ 2n/3.$$

Only votes with target exactly T contribute.

Definition 7 (Timeout)**.** On a chain with head state $\sigma[L]$ at height h, a *timeout* fires iff

$$\big|\{i : \text{timeouts}[i] = \texttt{true}\}\big| \ \geq\ 2n/3.$$

A timeout has no associated target block. Both timeout votes and height-fresh justification votes (with target $\prec L$ at the chain's current height) set timeouts$[i] =$ `true`, so a justification quorum implies a timeout quorum. The two height-advance branches are checked in order, justification first (Figure 1).

Definition 8 (Finality)**.** The justified block J is *finalized at height* h_j once $|P| \geq 2n/3$.

Definition 9 (Slashing, rule E1)**.** A validator is *slashable* if it signed two votes a, b (possibly $a = b$) with

$$b.\mathsf{finalize\_target} \neq \bot \ \wedge \ a.\mathsf{height} = b.\mathsf{finalize\_height} \ \wedge \ a.\mathsf{target} \neq b.\mathsf{finalize\_target}.$$

That is, a validator committing $(\mathsf{finalize\_height}, \mathsf{finalize\_target}) = (h_f, T_f)$ must not sign any vote at height h_f with target $\neq T_f$. Note that timeout votes ($a.\mathsf{target} = \bot$) are themselves in conflict with any commitment $b.\mathsf{finalize\_target} \neq \bot$ at the same height, since $\bot \neq b.\mathsf{finalize\_target}$.

Two justification votes (or two timeout votes) at the same height are not by themselves slashable; per-height uniqueness of honest justification votes is imposed behaviorally (Section 3). A validator who never sets $\mathsf{finalize\_target} \neq \bot$ trivially avoids slashing.

State transitions. The state machine (Figure 1) exposes one per-block entry point, stateTransition, plus the per-slot, per-block, per-height, and per-vote routines it invokes.

stateTransition first advances the slot counter from the parent state up to $B.\mathsf{slot}$ by iterating processSlot, then folds B's votes with processBlock, and finally calls processHeight on the post-vote state. Thus any cert event enabled by B's own votes fires in B's stored post-state, while any cert event enabled during a slot with no block fires from processSlot. After stateTransition returns, $\sigma.s = B.\mathsf{slot}$ and $\sigma.L = B$.

processSlot is the only routine that increments s. It calls processHeight before incrementing only when the current slot has no block on this chain ($\sigma.L.\mathsf{slot} < \sigma.s$); when $\sigma.L.\mathsf{slot} = \sigma.s$, the block at the current slot was already closed by stateTransition, so processSlot only advances the counter. processHeight runs the height-closing logic for the current slot or post-block state by invoking processHeightEvents, whose branches are checked in order: (i) finality, $F \leftarrow J$ if $|P| \geq 2n/3$; (ii) justification, if some T has $\geq 2n/3$ entries in targets; (iii) timeout cert, if $\geq 2n/3$ entries of timeouts are `true`. Branches (ii) and (iii) both advance height and reset targets, timeouts, so at most one height-advance fires per invocation of processHeightEvents.

processBlock assumes its caller has brought σ to slot $B.\mathsf{slot}$; it then sets $L \leftarrow B$ and feeds each vote v in B through processVote. A timeout vote ($v.\mathsf{target} = \bot$) at the current height sets $\mathsf{timeouts}[i] \leftarrow \mathtt{true}$. A height-fresh justification vote sets both $\mathsf{targets}[i] \leftarrow v.\mathsf{target}$ and $\mathsf{timeouts}[i] \leftarrow \mathtt{true}$. The P-gate adds i to P iff $v.\mathsf{finalize\_height} = \sigma.h_j$ and $v.\mathsf{finalize\_target} = \sigma.J$; this is independent of freshness.

Figure 1: State machine

```
1: function stateTransition(σ, B)                                   ▷ Per-block entry point
2:     while σ.s < B.slot do                                        ▷ Advance through intervening slots
3:         σ ← processSlot(σ)
4:     σ ← processBlock(σ, B)
5:     σ ← processHeight(σ)                                         ▷ Close events enabled by B's votes
6:     return σ
7: function processSlot(σ)
8:     if σ.L.slot < σ.s then                                       ▷ The current slot is empty on this chain
9:         σ ← processHeight(σ)
10:    σ.s ← σ.s + 1
11:    return σ
12: function processBlock(σ, B)
13:    assert σ.s = B.slot                                          ▷ Precondition: caller advanced σ.s
14:    σ.L ← B
15:    for each vote v in B do σ ← processVote(σ, v)
16:    return σ
17: function processVote(σ, v)
18:    i ← v.validator
19:    if v.target = ⊥ then                                         ▷ Timeout vote
20:        if v.height = σ.h then
21:            σ.timeouts[i] ← true
22:    else                                                         ▷ Justification vote: apply height freshness
23:        fresh ← (v.height = σ.h) ∧ (v.target ≺ σ.L) ∧ (v.target.slot ≥ σ.s_h)
24:        if fresh then
25:            σ.targets[i] ← v.target
26:            σ.timeouts[i] ← true
27:    if v.finalize_height = σ.h_j and v.finalize_target = σ.J then
28:        σ.P ← σ.P ∪ {i}
29:    return σ
30: function processHeight(σ)
31:    σ ← processHeightEvents(σ)
32:    return σ
33: function processHeightEvents(σ)
34:    if |σ.P| ≥ 2n/3 then                                         ▷ Finality
35:        σ.F ← σ.J
36:    T ← arg max_{T' ∈ {σ.targets[i] : σ.targets[i] ≠ ⊥}} |{i : σ.targets[i] = T'}|   ▷ T ← ⊥ if the set is empty
37:    if T ≠ ⊥ and |{i : σ.targets[i] = T}| ≥ 2n/3 then            ▷ Justification
38:        σ.J ← T,   σ.h_j ← σ.h,   σ.P ← ∅
39:        σ.h ← σ.h + 1,   σ.s_h ← σ.s
40:        reset targets, timeouts
41:        return σ
42:    if |{i : σ.timeouts[i] = true}| ≥ 2n/3 then                  ▷ Timeout
43:        σ.h ← σ.h + 1,   σ.s_h ← σ.s
44:        reset targets, timeouts
45:    return σ
46: function finalizeConflict(v_1, v_2)          ▷ Does v_1 conflict with v_2's finalize commitment?
47:    return v_2.finalize_target ≠ ⊥ ∧ v_1.height = v_2.finalize_height ∧ v_1.target ≠ v_2.finalize_target
48: function isSlashable(v_1, v_2)
49:    return v_1.validator = v_2.validator
           ∧ (finalizeConflict(v_1, v_2) ∨ finalizeConflict(v_2, v_1))
```

2 Accountable safety

Lemma 1 (Height progression)**.** *processHeight increments h by at most 1 per invocation, and each increment is gated by a $\geq 2n/3$ justification or timeout quorum.*

Proof. The only assignments to h are $h \leftarrow h + 1$ in the justification and timeout-cert branches of processHeightEvents, each guarded by its $\geq 2n/3$ quorum and followed by a return or function exit, so at most one fires per call. □

Lemma 2 (Freshness via target state-height)**.** *For any chain head L and ancestor $T \preceq L$,*

$$T.\,\text{slot} \geq \sigma[L].s_h \qquad \Longleftrightarrow \qquad \sigma[T].h = \sigma[L].h.$$

Proof. Let $h \coloneqq \sigma[L].h$. Along a chain, state-height only increases, and whenever a height-advance fires, processHeightEvents sets s_h to the slot boundary at which that advance fires. Thus $\sigma[L].s_h$ is exactly the boundary where L's chain entered state-height h. Any ancestor of L with slot before that boundary has state-height $< h$, while any ancestor at or after that boundary has state-height h. Applying this to $T \preceq L$ gives the equivalence. □

Lemma 3 (Checkpoint monotonicity)**.** *On any chain, the fields s, h, h_j, s_h, F, J are non-decreasing along $\preceq$, with $F \preceq J$ at all times. J and h_j advance only on justification events, and s_h only on height-advances, with s_h set to the slot boundary where the height advance fires.*

Proof. Reading off Figure 1: s only increases (in processSlot); h only increases, by 1, in the height-advance branches of processHeightEvents, which also set $h_j \leftarrow h$ on the justification branch and $s_h \leftarrow \sigma.s$ on either branch. Slots strictly increase along $\preceq$ (Definition 2), so s_h is non-decreasing. F updates only via $F \leftarrow J$, so $F \preceq J$ reduces to J-monotonicity.

Block field J. At a justification advance on chain Y firing at target T with pre-state height h, freshness forces $T.\,\text{slot} \geq s_h$, since every contributing justification vote has target T and was processed only when fresh. By Lemma 2, $\sigma[T].h = h$, so the previous justified block J_{old} with $\sigma[J_{\text{old}}].h \leq h - 1$ and $J_{\text{old}} \preceq Y$ satisfies $T \succ J_{\text{old}}$ by state-height monotonicity. Hence J strictly advances along $\preceq$ at every justification event. □

Lemma 4 (Any chain past a finalized height contains the finalized block)**.** *Unless $\geq n/3$ validators are slashable: if C is finalized at height h, then $C \preceq B$ for every block B with $\sigma[B].h > h$.*

Proof. By Lemma 1, some $B^* \preceq B$ advanced height from h to $h + 1$ via a quorum Q at height h on B^*'s chain (justification or timeout cert). Finality of C at h gives a quorum Q_F each signing a finalize-bearing vote b with $b.\text{finalize\_height} = h$ and $b.\text{finalize\_target} = C$. Quorum intersection gives $|Q \cap Q_F| \geq n/3$; pick $v \in Q \cap Q_F$ not slashable, with $b \in Q_F$ its finalize-bearing vote and a its contribution to Q at height h on B^*'s chain. Non-slashability rules out $\text{finalizeConflict}(a, b)$, so $a.\text{target} \neq \bot$ and $a.\text{target} = C$ (since $a.\text{height} = h = b.\text{finalize\_height}$ and $b.\text{finalize\_target} = C \neq \bot$). Hence a is a fresh justification vote (timeouts have $a.\text{target} = \bot$) processed on B^*'s chain at state-height h, and freshness gives $C = a.\text{target} \preceq B^* \preceq B$. □

Lemma 5 (Finalized blocks form a chain)**.** *Unless $\geq n/3$ validators are slashable: any two finalized checkpoints $(C, h), (C', h')$ are compatible, with $C \preceq C'$ whenever $h \leq h'$.*

Proof. Case $h = h'$. Finality of C gives a quorum Q_F with $\text{finalize\_height} = h$, $\text{finalize\_target} = C$. Finality of C' requires justification of C', giving a justification quorum Q' at height h with target C'. By quorum intersection, $|Q_F \cap Q'| \geq n/3$; pick $v \in Q_F \cap Q'$ not slashable. E1 (Definition 9) applied to v's votes in Q_F, Q' forces $C = C'$.

Case $h < h'$. Let Z be the chain on which C' was finalized, at the post-state where the finalize-quorum closed; then $\sigma[Z].h \geq h' > h$, $C' \preceq Z$, and $\sigma[C'].h = h'$ (Lemma 2). Lemma 4 at Z gives $C \preceq Z$; Lemma 2 on C's own finalizing chain gives $\sigma[C].h = h$. Both C, C' lie on Z, so comparable; with $\sigma[C].h = h < h' = \sigma[C'].h$, state-height monotonicity (Lemma 3) forces $C \prec C'$. □

Theorem 2 (Accountable safety)**.** *Unless $\geq n/3$ validators are slashable, no two conflicting blocks can be finalized.*

Proof. Immediate from Lemma 5: any two finalized blocks are compatible. □

3 Fork-choice store

A node may track multiple chains simultaneously; the *store* is the node-local data structure that mediates which chain the node follows. It maintains a single *root block* J — the block of the highest-key justification ever observed — together with that justification's height h_j. Each justification firing is compared against $(h_j, \mathsf{hash}(J))$ in lex order, and (J, h_j) is updated whenever a new justification strictly dominates. Because timeouts replace every prefix-notarization mechanism, the comparison uses the lexicographic pair $(h, \mathsf{hash}(C))$ associated with each justification firing (C, h): height first, hash tiebreak; under $f < n/3$ the tiebreak is rarely needed and exists for robustness only.

Definition 10 (Store)**.** A node maintains a *store*

$$\Sigma \;=\; (\sigma,\ \mathcal{T},\ F,\ J,\ h_j,\ h_{\max}),$$

where:

- σ is the *state map*, assigning each accepted block its per-chain post-state (Section 1);
- $\mathcal{T}$ is the *block tree*, the set of accepted blocks;
- F is the *store-finalized block*;
- J is the *store root*: the block of the highest-key justification ever observed;
- h_j is the height component of that justification key;
- $h_{\max} \in \mathbb{N}$ is the maximum state-height $\sigma[B].h$ over $B \in \mathcal{T}$.

The genesis store has $\mathcal{T} = \{B_{\text{gen}}\}$, $F = J = B_{\text{gen}}$, $h_j = 0$, $h_{\max} = 1$ (matching $\sigma[B_{\text{gen}}].h = 1$ from Definition 1). Section 5 specifies the concrete fork-choice walk (latest head votes, the fresh-quorum anchor with the grade-1 fallback, and the Goldfish layer) that instantiates the auxiliary Ω; the base store invariants below are unchanged.

The capital Σ distinguishes the store from the per-chain state σ. We write $\Sigma.\sigma$, $\Sigma.\mathcal{T}$, $\Sigma.F$, $\Sigma.J$, $\Sigma.h_j$, $\Sigma.h_{\max}$ for field access. The store key associated with the root is $(\Sigma.h_j, \mathsf{hash}(\Sigma.J))$.

Viable subtree. The store also defines a *viable subtree* $\mathcal{T}'(\Sigma) \subseteq \Sigma.\mathcal{T}$, gating which blocks getConfirmed may return.

Definition 11 (Viable subtree)**.** A *leaf* of $\Sigma.\mathcal{T}$ is a block with no proper descendant in $\Sigma.\mathcal{T}$. Define *viability* recursively:

- a leaf B is *viable* iff $\sigma[B].h \geq \Sigma.h_{\max} - 1$;
- an internal block B is viable iff some descendant of B in $\Sigma.\mathcal{T}$ is viable.

The *viable subtree* is $\mathcal{T}'(\Sigma) \coloneqq \{B \in \Sigma.\mathcal{T} : B \text{ is viable}\}$. Equivalently, $B \in \mathcal{T}'(\Sigma)$ iff some leaf $L \succeq B$ has $\sigma[L].h \geq \Sigma.h_{\max} - 1$.

The viability filter discards branches whose tip lags the most recently advanced chain by more than one height. For a block F finalized at height h_f, Lemma 4 under $f < n/3$ forces every block at σ-height strictly greater than h_f to descend from F, so when $h_{\max} > h_f$ any block at σ-height $h_{\max}$ descends from F (and when $h_{\max} = h_f$, F itself satisfies $\sigma[F].h = h_{\max} \geq h_{\max} - 1$); either way the filter is

automatic for finalized blocks. Blocks at σ-height $h_{\max} - 1$ may still conflict with F, since mainsafety constrains only strictly higher heights — this is why $\Sigma.J$ remains the primary walk-from block in the cascade whenever it has not slipped behind the frontier. $\Sigma.F$ itself is always viable (Lemma 7 below): the onBlock assertion $\Sigma.F \preceq B$ makes every newly admitted block a descendant of $\Sigma.F$, so the block driving any $\Sigma.h_{\max}$ bump is itself a viable witness for $\Sigma.F$.

Store mutators. The acceptance routine onBlock (Figure 2) admits B only if $B.\mathsf{parent} \in \Sigma.\mathcal{T}$ and $\Sigma.F \preceq B$. It invokes stateTransition($\Sigma.\sigma[B.\mathsf{parent}]$, B) (Figure 1), which advances through intervening slots, folds in B's votes, and closes height events enabled by the resulting post-vote state; the resulting post-state σ' becomes $\Sigma.\sigma[B]$. The post-state then offers the justified checkpoint $(\sigma'.J, \sigma'.h_j)$ to updateJustified and the finalized block $\sigma'.F$ to updateFinalized; $h_{\max}$ is bumped to $\max(\Sigma.h_{\max}, \sigma'.h)$ before either guard is evaluated, so the viability filter sees the new maximum.

updateJustified(Σ, J', h') first filters out any candidate J' with $J' \not\succeq \Sigma.F$ and then updates $(\Sigma.J, \Sigma.h_j)$ iff the candidate's justification key strictly exceeds $(\Sigma.h_j, \mathsf{hash}(\Sigma.J))$ in lex order; this maintains J incrementally as the lex-max justification ever observed on a chain extending the current F, never recomputed. updateFinalized(Σ, F') sets $\Sigma.F \leftarrow F'$ iff $F' \succ \Sigma.F$, $F' \preceq \Sigma.J$, *and* $F' \in \mathcal{T}'(\Sigma)$; the final clause is the *viability guard*, which prevents $\Sigma.F$ from settling at a block the height filter has rendered unreachable from getConfirmed.

getConfirmed(Σ, Ω) returns a block on the available chain that, under synchrony, is safe in the sense that it will not be reverted as long as more than half of the online validators are honest. Here Ω represents the state of the available chain — whatever extra information the validator uses to disambiguate among viable descendants of the walk-from block. The protocol fixes the walk-from rule via a two-way cascade: walk from $\Sigma.J$ when $\Sigma.h_{\max} = \Sigma.h_j + 1$, and otherwise walk from $\Sigma.F$ (always viable, Lemma 7). A justification at height h on a processed chain forces that chain's state-height to $h+1$, so $\Sigma.h_{\max} \geq \Sigma.h_j + 1$ always; the gate therefore holds precisely when no timeout has advanced any chain's state-height further past $\Sigma.J$'s height. When the gate holds, $\sigma[\Sigma.J].h = \Sigma.h_j = \Sigma.h_{\max} - 1$ meets the leaf-viability bound and $\Sigma.J \in \mathcal{T}'(\Sigma)$. When it fails, $\Sigma.J$ is "stale" — some chain has advanced past $\Sigma.J$'s height via timeout cert — and the adversary cannot force canonical to track such a justification even if its key dominates by lex. The choice among admissible descendants is left to Ω. Different validators may pass different Ω, and a single validator may pass different Ω at different call sites. Section 5 instantiates this Ω and the walk-from cascade concretely: Ω is the Goldfish available-chain state, the descent among viable descendants first fixes an anchor (a locally accepted fresh-root pointer, or the latest-head-vote grade-1 descent as fallback) and then follows Goldfish and the viability descent (Definition 21), and Lemma 17 shows the instantiation meets Corollary 1, so every store result of this section is preserved unchanged.

Figure 2: Store and fork-choice root

```
1: Genesis store Σ: Σ.𝒯 ← {B_gen}, Σ.F ← B_gen, Σ.J ← B_gen, Σ.h_j ← 0, Σ.h_max ← 1,
2:                  Σ.σ[B_gen] ← σ_gen
3: function onBlock(Σ, B)
4:     assert B.parent ∈ Σ.𝒯
5:     assert Σ.F ⪯ B
6:     Σ.𝒯 ← Σ.𝒯 ∪ {B}
7:     σ' ← stateTransition(Σ.σ[B.parent], B)
8:     Σ.σ[B] ← σ'
9:     Σ.h_max ← max(Σ.h_max, σ'.h)
10:    Σ ← updateJustified(Σ, σ'.J, σ'.h_j)
11:    Σ ← updateFinalized(Σ, σ'.F)
12:    return Σ
13: function updateJustified(Σ, J', h')                    ▷ F-filter, then running max
14:    if Σ.F ⪯ J' and (h', hash(J')) > (Σ.h_j, hash(Σ.J)) then
15:        Σ.J ← J';   Σ.h_j ← h'
16:    return Σ
17: function updateFinalized(Σ, F')
18:    if F' ≻ Σ.F and F' ⪯ Σ.J and F' ∈ 𝒯'(Σ) then
19:        Σ.F ← F'
20:    return Σ
21: function getConfirmed(Σ, Ω)
22:    R ← Σ.J if Σ.h_max = Σ.h_j + 1 else Σ.F              ▷ Σ.J is at the frontier
23:    return B ∈ 𝒯'(Σ) with B ⪰ R and σ[B].h ≥ Σ.h_max − 1, depending on Ω
```

The F-filter in updateJustified never discards a key-dominant event: Theorem 4 below shows that F advances only to blocks $\preceq J$, so every justification whose key could beat the current root remains on the F-chain.

3.1 Store invariants and safety

Unconditional invariants

Lemma 6 (h_j monotonicity)**.** *$\Sigma.h_j$ is non-decreasing over time, and the justification key $(\Sigma.h_j, \mathsf{hash}(\Sigma.J))$ is non-decreasing in lex order.*

Proof. Both $\Sigma.J$ and $\Sigma.h_j$ change only inside updateJustified, whose guard enforces that the new key strictly exceeds the old in lex order; in particular the height coordinate h_j is the leading component, so it is non-decreasing. □

Theorem 3 (Local irreversibility of finality)**.** *Once a node sets $\Sigma.F = F$, $\Sigma.F$ descends from F at all future times.*

Proof. $\Sigma.F$ is updated only by updateFinalized, whose guard $F' \succ \Sigma.F$ forces strict extension along $\preceq$. □

Theorem 4 ($F \preceq J$)**.** *The store maintains $\Sigma.F \preceq \Sigma.J$ at all times.*

Proof. By induction on store operations. At genesis, $F = J = B_{\text{gen}}$, so $F \preceq J$. Assume the invariant before a call to either mutator in Figure 2.

updateJustified. F is untouched, and J either stays or is replaced by a candidate J' that passed the $J' \succeq \Sigma.F$ filter; either way $J \succeq F$.

updateFinalized. J is untouched, and the guard $F' \preceq \Sigma.J$ ensures $F \leftarrow F' \preceq J$. □

Remark 5 (Store-finalized invariant). At all times $\Sigma.F$ is either B_{gen} or a block that was finalized on some chain; moreover, whenever a justification with target $J^\star$ has fired at height $h^\star$ on some chain processed by the node, the justification $(J^\star, h^\star)$ was offered to updateJustified once $J^\star \succeq \Sigma.F$ at the moment of offering. $\Sigma.F$ only advances via updateFinalized to $F' = \sigma[B].F$, and $\sigma[B].F \neq B_{\text{gen}}$ requires the finality branch of processHeightEvents to have fired on B's chain, so F' is a block finalized on that chain.

Remark 6 ($\mathcal{T}'$ shrinks only when $\Sigma.h_{\max}$ grows). Adding a block to $\Sigma.\mathcal{T}$ never removes any block from $\mathcal{T}'(\Sigma)$ unless the addition also increases $\Sigma.h_{\max}$. The viability threshold $\Sigma.h_{\max} - 1$ depends only on $\Sigma.h_{\max}$; while it is fixed, every previously viable leaf remains viable and every previously viable internal block retains its viable leaf witness, so $\mathcal{T}'(\Sigma)$ can only grow. Only an $\Sigma.h_{\max}$ bump can disqualify leaves whose σ-height falls below the new threshold.

Lemma 7 ($\Sigma.F$ is always viable). *$\Sigma.F \in \mathcal{T}'(\Sigma)$ at all times.*

Proof. At genesis, $\Sigma.F = B_{\text{gen}}$ and $\Sigma.h_{\max} = 1 = \sigma[B_{\text{gen}}].h \geq \Sigma.h_{\max} - 1$, so B_{gen} is a viable leaf. Inductively, $\Sigma.F$ changes only via updateFinalized, whose viability guard ensures the new value is viable. Block additions that do not bump $\Sigma.h_{\max}$ preserve $\Sigma.F$'s viability by Remark 6. The remaining case is an $\Sigma.h_{\max}$ bump in onBlock with new block B satisfying $\sigma[B].h > \Sigma.h_{\max}$; after the bump $\Sigma.h_{\max} = \sigma[B].h$, and the assertion $\Sigma.F \preceq B$ makes B — a leaf at the moment of addition with $\sigma[B].h$ equal to the new $\Sigma.h_{\max}$ — a viable leaf descendant of $\Sigma.F$, witnessing $\Sigma.F \in \mathcal{T}'(\Sigma)$. □

Corollary 1 (getConfirmed is total). *For every store Σ and auxiliary Ω, getConfirmed(Σ, Ω) returns a block in $\mathcal{T}'(\Sigma)$ at σ-height $\geq \Sigma.h_{\max} - 1$.*

Proof. If the cascade gate $\Sigma.h_{\max} = \Sigma.h_j + 1$ holds, $R = \Sigma.J$ with $\sigma[\Sigma.J].h = \Sigma.h_j = \Sigma.h_{\max} - 1$ (Lemma 2); state-height monotonicity along $\preceq$ (Lemma 3) gives every descendant of $\Sigma.J$ in $\Sigma.\mathcal{T}$ a σ-height $\geq \Sigma.h_{\max} - 1$, so any leaf of $\Sigma.\mathcal{T}$ above $\Sigma.J$ is a viable leaf and witnesses $\Sigma.J \in \mathcal{T}'(\Sigma)$. Otherwise $R = \Sigma.F$, and Lemma 7 gives a viable leaf descending from $\Sigma.F$ in $\Sigma.\mathcal{T}$. □

Theorem 7 (Fork-choice consistency). *Once a node sets $\Sigma.F = F$, getConfirmed(Σ, Ω) returns a block descending from F at all future times, for every Ω.*

Proof. By Theorems 3 and 4, $F \preceq \Sigma.F \preceq \Sigma.J$ at all future times. Both branches of getConfirmed return a descendant of $\Sigma.J$ or $\Sigma.F$ (Corollary 1); either descends from F. □

Further properties under fewer than $n/3$ slashable validators

Lemma 8 (Justification chain). *Unless $\geq n/3$ validators are slashable: if F is finalized at height h_f and a justification (C, h) has fired on any chain processed by the node with $h \geq h_f$, then F and C are compatible, with $F \prec C$ when $h > h_f$.*

Proof. If $h_f = 0$ then $F = B_{\text{gen}}$ and compatibility is trivial; assume $h_f \geq 1$. The justification fired at the post-state of some processed block B whose onBlock call triggered the justification branch of processHeightEvents with $(\sigma[B].J, \sigma[B].h_j) = (C, h)$, so $C \preceq B$ and $\sigma[B].h \geq h + 1$ (justification advances height). Lemma 4 at $\sigma[B].h > h_f$ gives $F \preceq B$, and with $C \preceq B$ this yields $F \sim C$. For $h > h_f$: $\sigma[F].h = h_f < h = \sigma[C].h$ by Lemma 2, and state-height monotonicity along $\preceq$ (Lemma 3) forces $F \prec C$ on the comparable pair. □

Lemma 9 (Upgrade property). *Unless $\geq n/3$ validators are slashable: if F is finalized at height h_f and some block B with $\sigma[B].J = F$ and $\sigma[B].h_j = h_f$ has been processed by the node, then $\Sigma.J \succeq F$ at all future times.*

Proof. Let $F^0 \coloneqq \Sigma.F$ at the moment $\mathsf{onBlock}(B)$ offered (F, h_f) to $\mathsf{updateJustified}$. Both F^0 and F are genesis or finalized (Remark 5), so Lemma 5 makes them comparable. If $F \preceq F^0$, Theorem 4 gives $\Sigma.J \succeq \Sigma.F \succeq F^0 \succeq F$ at that moment, and Theorems 3 and 4 preserve $\Sigma.J \succeq \Sigma.F \succeq F$ thereafter. Otherwise $F^0 \prec F$ strictly: the F-filter $\Sigma.F \preceq F$ passes, so the call brings $(\Sigma.h_j, \mathsf{hash}(\Sigma.J)) \geq (h_f, \mathsf{hash}(F))$ in lex order, preserved by Lemma 6. At any later moment, $\Sigma.h_j \geq h_f$, and the current $\Sigma.J$ is justified at some $h' \geq h_f$ on a processed chain: if $h' > h_f$, Lemma 8 gives $F \prec \Sigma.J$; if $h' = h_f$, the justification quorum Q' for $\Sigma.J$ at h_f and the finality quorum Q_F for F at h_f intersect in $\geq n/3$, and a non-slashable $v \in Q' \cap Q_F$ forces $\Sigma.J = F$ by E1. Either way $\Sigma.J \succeq F$. □

Lemma 10 (Finalized blocks are viable)**.** *Unless $\geq n/3$ validators are slashable: if F is finalized at height h_f and some block B with $\sigma[B].J = F$ and $\sigma[B].h_j = h_f$ has been processed by the node, then $F \in \mathcal{T}'(\Sigma)$ at all future times.*

Proof. Processing of B with $\sigma[B].h_j = h_f$ means the justification branch of $\mathsf{processHeightEvents}$ fired with target F at pre-state height h_f on B's chain, advancing σ to $\sigma[B].h \geq h_f + 1$; hence $\Sigma.h_{\max} \geq h_f + 1 > h_f$ at all future times. Pick any leaf $L \in \Sigma.\mathcal{T}$ with $\sigma[L].h = \Sigma.h_{\max}$ (some block achieves $\Sigma.h_{\max}$ by definition, and any leaf descendant preserves the maximum by state-height monotonicity). Then L is viable, and Lemma 4 gives $F \preceq L$, so L witnesses $F \in \mathcal{T}'(\Sigma)$. □

Theorem 8 (Local acceptance of finality updates)**.** *Unless $\geq n/3$ validators are slashable: if a block B is processed by $\mathsf{onBlock}$ and $\sigma[B].F = F'$, then after processing $\Sigma.F \succeq F'$.*

Proof. If $\Sigma.F \succeq F'$ already, processing B preserves the bound by monotonicity of $\Sigma.F$ (Theorem 3). Otherwise $\Sigma.F$ is genesis or finalized (Remark 5), so Lemma 5 gives $\Sigma.F \sim F'$ and hence $\Sigma.F \prec F'$. $\sigma[B].F = F'$ requires some ancestor $B' \preceq B$ with $\sigma[B'].J = F'$ and $\sigma[B'].h_j = h_{F'}$; B' was processed before B (parent-first), so Lemma 9 gives $\Sigma.J \succeq F'$, hence the $F' \preceq \Sigma.J$ guard passes. For the viability guard, Lemma 10 gives $F' \in \mathcal{T}'(\Sigma)$. All three guards pass, so $\Sigma.F \leftarrow F'$. □

Theorem 9 (Lock-in)**.** *Unless $\geq n/3$ validators are slashable: if block F is finalized at height h_f on any chain, and some block B with $\sigma[B].J = F$ and $\sigma[B].h_j = h_f$ has been processed by the node, then $\Sigma.J \succeq F$ at all future times, $F \in \mathcal{T}'(\Sigma)$ at all future times, and $\mathsf{getConfirmed}(\Sigma, \Omega)$ always returns a descendant of F, for every Ω.*

Proof. $\Sigma.J \succeq F$ by Lemma 9, so $\sigma[\Sigma.J].h = \Sigma.h_j \geq h_f$ by state-height monotonicity along $F \preceq \Sigma.J$ (using $\sigma[F].h = h_f$ from Lemma 2); $F \in \mathcal{T}'(\Sigma)$ by Lemma 10. If $\Sigma.h_{\max} = \Sigma.h_j + 1$, $\mathsf{getConfirmed}$'s first branch fires and returns a descendant of $\Sigma.J \succeq F$. Otherwise $\Sigma.h_{\max} \geq \Sigma.h_j + 2 \geq h_f + 2$, and $\mathsf{getConfirmed}$'s second branch returns a block T with $\sigma[T].h \geq \Sigma.h_{\max} - 1 \geq h_f + 1 > h_f$, so Lemma 4 gives $F \preceq T$. □

Theorem 10 (Order independence)**.** *Unless $\geq n/3$ validators are slashable: the observable store view after folding the same available set of blocks through $\mathsf{onBlock}$ in any parent-first order depends only on that set, not on the order. In particular, two nodes with the same available blocks agree on $(F, J, h_j, h_{\max})$, on the accepted subtree rooted at F, and hence on the possible outputs of $\mathsf{getConfirmed}$. They need not agree on blocks outside the subtree of F: a block conflicting with the final F may have been accepted before finality moved at one node, while another node that first moved F would reject it.*

Proof. Fix a common input set $\mathcal{B}$ and consider any parent-first processing order. For every block whose ancestors are in $\mathcal{B}$, its post-state is a deterministic function of itself and its parent's post-state; write this state as $\sigma[B]$.

F. By Lemma 5, $\{\sigma[B].F : B \in \mathcal{B}\}$ is a chain with maximum $F_{\max}$, attained at some $B^* \in \mathcal{B}$. At all times, $\Sigma.F \preceq F_{\max}$. When B^* is processed, Theorem 8 gives $\Sigma.F \succeq F_{\max}$; monotonicity of $\Sigma.F$ then gives final value $\Sigma.F = F_{\max}$ in every order.

$h_{\max}$. Let $B_{\max}$ maximize $\sigma[B].h$ over $\mathcal{B}$. If $\sigma[B_{\max}].h > h_{F_{\max}}$, Lemma 4 gives $F_{\max} \preceq B_{\max}$. If $\sigma[B_{\max}].h = h_{F_{\max}}$, then $F_{\max}$ itself attains the same maximum height. Thus some maximum-height

block descends from $F_{\max}$. Since every intermediate finalized root is below $F_{\max}$, the onBlock finality-ancestor assertion accepts such a maximum-height block whenever it is reached in a parent-first order. Thus $\Sigma.h_{\max}$ reaches the maximum σ-height over $\mathcal{B}$, and no processed block can raise it further.

J. For determining the final justified root, only cert descriptors with height at least $h_{F_{\max}}$ can matter. Such descriptors are deterministic from $\mathcal{B}$: if a block's descriptor is (C, h) with $h \geq h_{F_{\max}}$, then the argument below shows $F_{\max} \preceq C$, and since C is an ancestor of the block producing the descriptor, that block lies in the live subtree and is accepted in every order. Let D be this deterministic high-descriptor multiset. Write $K_{\max} \coloneqq (h_{F_{\max}}, \mathsf{hash}(F_{\max}))$. For any $(C, h) \in D$ with $h \geq h_{F_{\max}}$: if $h > h_{F_{\max}}$, Lemma 8 gives $C \succ F_{\max}$; if $h = h_{F_{\max}}$, finality of $F_{\max}$ at $h_{F_{\max}}$ provides a finality quorum Q_F of size $\geq 2n/3$ with finalize_height $= h_{F_{\max}}$ and finalize_target $= F_{\max}$, while the descriptor's existence gives a justification quorum Q' at height $h_{F_{\max}}$ targeting C; quorum intersection picks a non-slashable $v \in Q' \cap Q_F$, and E1 (as in Lemma 9) forces $C = F_{\max}$. Descriptors with $h < h_{F_{\max}}$ have key strictly below $K_{\max}$ on the leading coordinate, so once $\Sigma.h_j \geq h_{F_{\max}}$ holds they are dominated. Hence $\Sigma.J$'s final value is the lex-max over $\{(C, h) \in D : h \geq h_{F_{\max}}\}$, a deterministic function of the processed set.

Live subtree and outputs. If $B \in \mathcal{B}$ and $F_{\max} \preceq B$, then every ancestor of B above $F_{\max}$ also descends from every intermediate $\Sigma.F$. Thus B is accepted in every parent-first order. Conversely, blocks outside the subtree of $F_{\max}$ may differ across nodes, but getConfirmed only returns descendants of the confirmation root, which is either $F_{\max}$ or $J \succeq F_{\max}$, and applies the same height threshold $h_{\max} - 1$. Therefore the possible outputs of getConfirmed are order-independent. □

Lemma 11 (Justifications stay at or below $\Sigma.h_j$)**.** *Unless $\geq n/3$ validators are slashable, every justification (C, h) that has fired on any chain processed by the node satisfies $h \leq \Sigma.h_j$ at all subsequent times.*

Proof. The justification fires inside an onBlock call on some processed block B, at which point Remark 5 gives the descriptor (C, h) offered to updateJustified. C lies on B's chain (it is the justified target), and so does $\Sigma.F$ (by $\Sigma.F \preceq B$), so $\Sigma.F$ and C are comparable. If $\Sigma.F \not\preceq C$, then $\Sigma.F \succ C$, so $h_f = \sigma[\Sigma.F].h \geq \sigma[C].h = h$ by state-height monotonicity (Lemma 3, applied along $C \prec \Sigma.F$, using $\sigma[C].h = h$ from Lemma 2). Combining $\Sigma.F \preceq \Sigma.J$ (Theorem 4) with $\sigma[\Sigma.J].h = \Sigma.h_j$ (Lemma 2) and the same state-height monotonicity along $\Sigma.F \preceq \Sigma.J$ gives $\Sigma.h_j \geq h_f \geq h$. Otherwise the F-filter passes and updateJustified's lex test forces $\Sigma.h_j \geq h$ after the call. Lemma 6 preserves the bound at all later times. □

4 Inactivity leak

This section extends the per-chain state machine of Section 1 with a penalty counter and a new transition function processInactivityPenalties, invoked by an extended processHeight (Figure 3); processSlot and stateTransition pick up the extension transparently through their calls to processHeight, while processBlock, processHeightEvents, and processVote are unchanged. Write $T = \mathsf{getConfirmed}(\Sigma, \Omega)$ for the *canonical chain*, with Ω the validator's auxiliary state.

Two results follow. Theorem 12 is purely state-logic: on any fixed chain, the total penalty increment over a non-finality period is at least $(T - 1) \cdot n/3$, with no hypothesis on synchrony, honesty, or the slashable set. Theorem 13 is local to a single honest i: the vote i produces per Definition 14, once included on any extension of i's canonical chain still at the same state-height, exempts i from Layer 1 penalties at that state-height. Layer 2 exemption is out of scope: it depends on quorum formation on a common target, not on any local property of i.

4.1 Penalty units

Augment the per-chain state tuple with one non-negative counter $\sigma.\mathsf{penalties}[1..n]$, initialized to 0 at genesis; no other field is added or modified. Figure 3 replaces processHeight from Section 1

with a version that snapshots the pre-advance state and invokes processInactivityPenalties after processHeightEvents. The store of Section 3 invokes the extended routine through the same stateTransition entry point.

Figure 3: Penalty-tracking state extension (leak section)

```
1:  function processHeight(σ)                                          ▷ extended: tracks penalties
2:      σ_pre ← σ                                                      ▷ snapshot of pre-advance state
3:      σ ← processHeightEvents(σ)
4:      σ ← processInactivityPenalties(σ_pre, σ)
5:      return σ
6:  function processInactivityPenalties(σ_pre, σ_post)
7:      if σ_pre.h = σ_post.h then                                     ▷ Layer 1: stall
8:          for each i ∈ {1, ..., n} do
9:              if σ_post.timeouts[i] = false then
10:                 σ_post.penalties[i] ← σ_post.penalties[i] + 1
11:     if σ_pre.J = σ_post.J then          ▷ Layer 2 target: justification did not advance
12:         T* ← arg max_{T' ∈ {σ_pre.targets[i] : σ_pre.targets[i] ≠ ⊥}} |{i : σ_pre.targets[i] = T'}|
13:         for each i ∈ {1, ..., n} do
14:             if σ_pre.targets[i] ≠ T* or σ_pre.targets[i] = ⊥ then
15:                 σ_post.penalties[i] ← σ_post.penalties[i] + 1
16:     if σ_pre.F = σ_post.F and σ_pre.F ≺ σ_pre.J then               ▷ Layer 2 finalize
17:         for each i ∈ {1, ..., n} do
18:             if i ∉ σ_pre.P then
19:                 σ_post.penalties[i] ← σ_post.penalties[i] + 1
20:     return σ_post
```

Reading Figure 3. processInactivityPenalties checks three independent guards; any subset may fire at a given slot.

Layer 1 (stall). Fires when the slot did not advance height. Every i with $\sigma_{\text{post}}.\mathsf{timeouts}[i] = \texttt{false}$ is bumped, i.e. every validator that failed to cast a vote setting the timeout marker at the current height (timeouts is not reset at a stall). The marker is set by either a timeout vote (target $\perp$) at the current height or a height-fresh justification vote with target on this chain (Section 1).

Layer 2 target. Fires when the justification branch of processHeightEvents did not fire (else J would strictly advance by Lemma 3). T^* is the plurality target across $\sigma_{\text{pre}}.\mathsf{targets}$ (deterministic tiebreak), or $\perp$ if no justification votes have been registered; the second disjunct of the guard bumps non-voters when $T^* = \perp$ (in which case all n validators are bumped). Since no $2n/3$-quorum on a single target formed, the plurality count is $< 2n/3$, so $|\{i : \sigma_{\text{pre}}.\mathsf{targets}[i] \neq T^* \vee \sigma_{\text{pre}}.\mathsf{targets}[i] = \perp\}| > n/3$.

Layer 2 finalize. Fires when a pending-finality gap is open at the start of the slot ($\sigma_{\text{pre}}.F \prec \sigma_{\text{pre}}.J$) and F did not advance; every $i \notin \sigma_{\text{pre}}.P$ is bumped. The finality branch of processHeightEvents, when it fires, sets $\sigma_{\text{post}}.F = \sigma_{\text{pre}}.J \succ \sigma_{\text{pre}}.F$, so this guard is skipped automatically.

Remark 11 (The empty vote and the Layer-1 guard). An honest validator gate-blocked from *both* a target and a timeout vote at a height (Definition 28) — its safe-confirmed head has not reached the voted interval — emits the empty vote Checkpoint() (Definition 29): a vote with an empty voted checkpoint (target $= \perp$, height $= \perp$) carrying a populated head field. Because it makes no current-height claim it leaves $\mathsf{timeouts}[i] = \texttt{false}$, so the Layer 1 guard above *does* bump i: unlike a timeout vote, an empty vote is not Layer-1-exempt. Its head field still carries i's SG latest message (Section 5.1) and its finalize fields still feed P, so it remains neutral for the Layer-2 finalize guard; only the Layer-

1 stall guard sees it as a non-participant. This is deliberate: it keeps the empty vote out of the timeout certificate, so that reaching $H+2$ from a justifiable height always certifies a safe confirmation (Lemma 23). Since the empty vote is now cast at *every* gated-out height (Definition 29), the exposure applies uniformly, not only during recovery. The resulting Layer-1 exposure is negligible under the healing dynamics — gate-blocked silence is sub-epoch while the leak accrues per epoch — and a leak-neutral per-validator presence-field variant is listed as an alternative; see Remark 15. See the gate (Definition 28) and Theorem 26(a) for how gate-blocking and the empty vote interact during healing.

4.2 Leak tightness

Definition 12 (Non-finality period)**.** Fix a chain and write $\sigma^{(t)}_{\text{pre}}, \sigma^{(t)}_{\text{post}}$ for its pre/post-processHeight snapshots at slot t. A *non-finality period* is a maximal interval $[t_1, t_T]$ of consecutive slots on which F is stuck, i.e., $\sigma^{(t)}_{\text{post}}.F = \sigma^{(t)}_{\text{pre}}.F$ throughout. Each such slot falls into exactly one category:

- *pending-stuck*: $\sigma^{(t)}_{\text{pre}}.F \prec \sigma^{(t)}_{\text{pre}}.J$ (pending-finality gap open at the start);
- *stable-stuck*: $\sigma^{(t)}_{\text{pre}}.F = \sigma^{(t)}_{\text{pre}}.J$ and J does not advance at t;
- *transition*: $\sigma^{(t)}_{\text{pre}}.F = \sigma^{(t)}_{\text{pre}}.J$ and the justification branch fires at t (opening a gap by slot's end).

Theorem 12 (Non-finality period penalty bound)**.** *Over any non-finality period of length $T \geq 1$ on any chain, the total penalty increment across all validators assessed by processInactivityPenalties on that chain is at least $(T-1) \cdot n/3$.*

Proof. Fix a non-finality period $[t_1, t_T]$. We show (i) at most one slot is transition, and (ii) every pending-stuck or stable-stuck slot contributes $> n/3$ to the total penalty increment.

(i) At most one transition slot. After a transition at t, $\sigma^{(t)}_{\text{post}}.J \succ \sigma^{(t)}_{\text{pre}}.F$; since F does not advance in the period and J is monotone across slots, $\sigma^{(t')}_{\text{pre}}.F \prec \sigma^{(t')}_{\text{pre}}.J$ at every later $t' > t$, making every such t' pending-stuck.

(ii) Per-slot charge $> n/3$.

Pending-stuck. The Layer 2 finalize guard fires at t. The finality branch of processHeightEvents did not fire (else F would advance), so $|\sigma^{(t)}_{\text{pre}}.P| < 2n/3$ and $|\{i : i \notin \sigma^{(t)}_{\text{pre}}.P\}| > n/3$.

Stable-stuck. The Layer 2 target guard fires at t. The justification branch did not fire, so the plurality count on $\sigma^{(t)}_{\text{pre}}$.targets is $< 2n/3$ (counting $T^* = \perp$ as 0); hence the bumped set has size $> n/3$.

At most one slot is transition, so at least $T-1$ slots are pending-stuck or stable-stuck; each contributes $> n/3$ by (ii), and Layer 1 adds only non-negative charges. Summing gives total increment $\geq (T-1) \cdot n/3$. □

4.3 Honest voting rule

We state the honest rule explicitly, as it underpins the leak fairness proof. The rule consumes three pieces of validator-local state: the store Σ (drives fork-choice), the auxiliary Ω (disambiguates among admissible getConfirmed outputs), and the validator's *local signing history* Δ (a private summary of its own past signatures, used to avoid slashing).

Definition 13 (Local signing history)**.** The local signing history Δ associates to each height h a triple $(\Delta.T(h), \Delta.\tau(h), \Delta.\lambda(h)) \in (\text{Block} \cup \{\perp\}) \times \texttt{bool} \times \texttt{bool}$, initialized to $(\perp, \texttt{false}, \texttt{false})$: $\Delta.T(h)$ is $\perp$ until i's first non-$\perp$ vote at h, after which it pins the target of that vote; $\Delta.\tau(h)$ is true once i has voted timeout at h; and $\Delta.\lambda(h)$ is true once i has voted to finalize $\Delta.T(h)$ at height h (i.e. has cast a finalize-bearing vote with finalize_height $= h$ and finalize_target $= \Delta.T(h)$). Δ is updated only by rememberVote (Figure 4), invoked once per signed vote. The honest reachable per-height configurations are $(\perp, \texttt{false}, \texttt{false})$, $(T, \texttt{false}, \texttt{false})$, $(T, \texttt{false}, \texttt{true})$, and $(T, \texttt{true}, \texttt{false})$; the

configuration $(T, \texttt{true}, \texttt{true})$ is the slashable trace (a finalize-bearing vote at h alongside a timeout vote at h).

Implementation. Δ may safely drop entries at heights below $\Sigma.h_{\max} - 1$; getFinalVote then conservatively clears the finalize commitment at older heights (because the gate $\Delta.T(h_f) = T_f$ fails on a missing entry), but every emitted vote remains E1-safe (Lemma 12). Δ is the entire state needed for safe vote signing on the validator-client side, depending only on Δ, separated from fork-choice on the beacon-client side (depending on Σ and Ω).

Definition 14 (Honest voting rule)**.** At each voting opportunity, i produces its vote by getFinalVote(Δ, getBaseVote(Σ, Ω)) (Figure 4) and signs the result. getBaseVote reads fork-choice state to assemble an unconstrained candidate; getFinalVote adjusts it to respect Δ's prior commitments and folds the new vote back into Δ via rememberVote.

Figure 4: Honest voting rule

```
1:  function getBaseVote(Σ, Ω)                                   ▷ the unconstrained vote
2:      T ← getConfirmed(Σ, Ω)
3:      h_c ← σ[T].h;   h_f ← σ[T].h_j;   T_f ← σ[T].J
4:      return (i, h_c, T, h_f, T_f)
5:  function getFinalVote(Δ, v)
6:      (i, h_c, T, h_f, T_f) ← v
7:      assert h_c > h_f                  ▷ vote-height strictly above the finalize-commitment height
8:      if Δ.λ(h_c) = true then                          ▷ lock at h_c: re-submit lock target
9:          T ← Δ.T(h_c)
10:     else if Δ.T(h_c) ≠ ⊥ then                  ▷ voted at h_c before, not locked: timeout
11:         T ← ⊥
12:     if Δ.T(h_f) ≠ T_f or Δ.τ(h_f) = true then          ▷ cannot E1-safely commit to finalize
13:         (h_f, T_f) ← (⊥, ⊥)
14:     v' ← (i, h_c, T, h_f, T_f)
15:     rememberVote(Δ, v')
16:     return v'
17: function rememberVote(Δ, v)
18:     h ← v.height;   h_f ← v.finalize_height
19:     if v.target ≠ ⊥ and Δ.T(h) = ⊥ then
20:         Δ.T(h) ← v.target
21:     else if v.target = ⊥ then
22:         Δ.τ(h) ← true
23:     if v.finalize_target ≠ ⊥ then
24:         Δ.λ(h_f) ← true
```

The target adjustment in getFinalVote produces the lock target when i is locked at h_c; a timeout when i has voted at h_c without locking (so a second non-$\perp$ vote at h_c would violate per-height target consistency); or the unconstrained T otherwise (first vote at h_c). The finalize fields are kept iff the gate $\Delta.T(h_f) = T_f \wedge \Delta.\tau(h_f) = \texttt{false}$ holds (so i previously voted T_f at h_f and has not voted timeout there); the assert $h_c > h_f$ rules out the degenerate case where the vote and the commitment would land at the same height (which would be E1-self-slashable when $T \neq T_f$). The gate is the same in all three target branches, so locks attach finalize symmetrically with first-votes whenever the gate passes.

Lemma 12 (Honest E1 safety)**.** *If validator i follows Definition 14, then i never produces a vote-pair (v_1, v_2) with $v_2.\mathsf{finalize\_target} \neq \bot$, $v_1.\mathsf{height} = v_2.\mathsf{finalize\_height}$, and $v_1.\mathsf{target} \neq v_2.\mathsf{finalize\_target}$.*

Proof. Write $h_* \coloneqq v_2.\mathsf{finalize\_height}$ and $T_* \coloneqq v_2.\mathsf{finalize\_target} \neq \bot$. v_2 retained finalize fields, so the gate inside getFinalVote held at v_2's firing: $\Delta.T(h_*) = T_* \neq \bot$, $\Delta.\tau(h_*) = \texttt{false}$, and $v_2.\mathsf{target} \neq \bot$; the subsequent rememberVote then set $\Delta.\lambda(h_*) \leftarrow \texttt{true}$. Since $v_2.\mathsf{target} \neq \bot$, the target-adjustment in getFinalVote did not enter its timeout branch, so $v_2.\mathsf{target} = T$ (first-vote) or $v_2.\mathsf{target} = \Delta.T(h_c)$ (lock); either way $v_2.\mathsf{target} \neq \bot$ at $h_c = v_2.\mathsf{height}$, while $\Delta.T(h_*) = T_* \neq \bot$ at $h_* = v_2.\mathsf{finalize\_height}$.

If $v_1.\mathsf{height} = h_*$ and v_1 was cast before v_2: $\Delta.\tau$-monotonicity and $\Delta.\tau(h_*) = \texttt{false}$ at v_2's firing rule out v_1 being a timeout at h_*, so $v_1.\mathsf{target} \neq \bot$. $\Delta.T(h_*)$ is set once at i's first non-$\bot$ vote at h_* and pinned thereafter, and the target adjustment in getFinalVote forces every subsequent non-$\bot$ vote of i at h_* to equal $\Delta.T(h_*)$ (lock branch returns $\Delta.T(h_*)$; first-vote branch is unreachable once $\Delta.T(h_*) \neq \bot$). Hence $v_1.\mathsf{target} = \Delta.T(h_*) = T_*$.

If $v_1.\mathsf{height} = h_*$ and v_1 was cast after v_2: at v_1's firing $\Delta.\lambda(h_*) = \texttt{true}$ (set by v_2), so the lock branch fires and $v_1.\mathsf{target} = \Delta.T(h_*) = T_*$ ($\Delta.T(h_*)$ is fixed after v_2's firing).

If $v_1 = v_2$: $v_1.\mathsf{height} = v_2.\mathsf{height} = h_c$, so the hypothesis $v_1.\mathsf{height} = v_2.\mathsf{finalize\_height}$ requires $h_c = h_*$. At genesis ($h_c = h_j = 0$) this collapses with $J = B_{\text{gen}}$: $v_2.\mathsf{target} = B_{\text{gen}}$ in either the first-vote or lock branch ($\Delta.T(0)$ is either $\bot$, in which case the unconstrained $T = B_{\text{gen}}$, or already B_{gen}, the only block at σ-height 0); hence $v_1.\mathsf{target} = v_2.\mathsf{target} = B_{\text{gen}} = T_*$. Post-genesis, every height advance sets $h_j \leftarrow h$ before $h \leftarrow h + 1$, giving $h_c \geq h_j + 1 > h_*$ at any state where v_2 could fire, so $h_c = h_*$ does not occur. □

Lemma 13 (Per-height justification uniqueness)**.** *If $f < n/3$ and all honest validators follow Definition 14: for any two blocks C_1, C_2 justified at the same height h on any chains processed by the node, $C_1 = C_2$.*

Proof. For $h = 0$, the only "justification" is the genesis pre-justification convention ($\sigma[B_{\text{gen}}].J = B_{\text{gen}}$, $h_j = 0$), which is unique by construction; the rest of the proof handles $h \geq 1$, where a $\geq 2n/3$ quorum is required.

Each justification at height h on a chain yields a quorum Q_k of size $\geq 2n/3$; quorum intersection picks an honest $v \in Q_1 \cap Q_2$ contributing a non-$\bot$-target vote at height h to each. Per Definition 13 and the target adjustment in getFinalVote, every non-$\bot$-target vote of v at height h has target equal to $\Delta.T(h)$ (set once at v's first such vote and pinned thereafter; subsequent non-$\bot$ votes at h come from the lock branch, which returns $\Delta.T(h)$). Hence both contributions have the same target, $C_1 = C_2$. □

4.4 Leak fairness

Fairness is local to a single honest validator i: we fix i, its local store Σ, its auxiliary state Ω, and the canonical chain $T = \mathsf{getConfirmed}(\Sigma, \Omega)$. The claim is that the vote v produced by Definition 14, once included on any extension of T still at the same state-height, exempts i from Layer 1 penalties at that state-height. Height advances reset targets and timeouts (Section 1), so Layer 2 state at the next height is independent of i's prior votes and no "lock" issue propagates.

The proof rests on two facts about the canonical chain. First, $T = \mathsf{getConfirmed}(\Sigma, \Omega)$ has σ-height $\geq \Sigma.h_{\max} - 1$ by the algorithm spec, so $h_c = \sigma[T].h \in \{\Sigma.h_{\max} - 1, \Sigma.h_{\max}\}$. Second, every locally observed justification at height h is witnessed by some processed block whose post-state advanced to σ-height $h+1$, so $h \leq \Sigma.h_{\max} - 1$. Together: at $h_c = \Sigma.h_{\max}$ this node has not observed a justification at h_c, so the lock branch is unreachable; at $h_c = \Sigma.h_{\max} - 1$ the lock branch forces the lock target to coincide with $\Sigma.J$ via Lemma 13, and $\Sigma.J$ is itself viable (its witnessing block sits at σ-height $\Sigma.h_{\max}$ as a descendant of $\Sigma.J$).

Lemma 14 (Honest vote sets the timeout marker)**.** *Assume $f < n/3$ and all honest validators follow Definition 14. Let i be honest with local store Σ and auxiliary state Ω, $T = \mathsf{getConfirmed}(\Sigma, \Omega)$, and $h_c = \sigma[T].h$. Let v be the vote i produces at the current slot. Then either $v.\mathsf{target} = \bot$ with $v.\mathsf{height} = h_c$, or $v.\mathsf{target} \preceq T$ with $\sigma[v.\mathsf{target}].h = h_c$.*

Proof. By Definition 14, $v.\mathsf{height} = h_c$ in every branch. The elif-timeout branch ($\Delta.T(h_c) \neq \bot$ with $\Delta.\lambda(h_c) = \texttt{false}$) emits $v.\mathsf{target} = \bot$, satisfying the first disjunct. The first-vote-at-h_c branch sets $v.\mathsf{target} = T$ with $\sigma[T].h = h_c$ and $T \preceq T$, satisfying the second disjunct.

The remaining branch is the lock branch. Let $k \coloneqq v_0.\mathsf{finalize\_height}$ for the earlier finalize-bearing vote v_0 of i whose rememberVote call set $\Delta.\lambda(k) \leftarrow \texttt{true}$; the lock-branch firing condition $\Delta.\lambda(h_c) = \texttt{true}$ at the current slot requires $k = h_c$, so the current vote's h_c equals v_0's finalize-height. Write $T_{\text{lock}} \coloneqq v_0.\mathsf{finalize\_target}$. For v_0 to retain its finalize fields, the gate inside getFinalVote had to hold at v_0's firing: $\Delta.T(k) = v_0.\mathsf{finalize\_target} = T_{\text{lock}}$ (with $\Delta.\tau(k) = \texttt{false}$). Since $\Delta.T(k)$ is pinned once set (cf. the target adjustment in getFinalVote), it remains T_{lock} at v's firing, and the lock branch returns $v.\mathsf{target} = \Delta.T(h_c) = T_{\text{lock}}$. Since v_0 is finalize-bearing, getBaseVote populated its finalize fields from i's then-canonical state, giving $(v_0.\mathsf{finalize\_height}, v_0.\mathsf{finalize\_target}) = (k, T_{\text{lock}}) = (\sigma[T^0].h_j, \sigma[T^0].J)$ at v_0's firing, where T^0 is i's canonical chain at that earlier time. The justification branch of processHeightEvents thus fired with target T_{lock} at pre-state height k inside some onBlock call on a processed block $B^\star$, whose post-state has $\sigma[B^\star].h \geq k{+}1$ (height-advance). Hence $\Sigma.h_{\max} \geq k{+}1 = h_c{+}1$, i.e., $h_c \leq \Sigma.h_{\max} - 1$.

Since $T = \mathsf{getConfirmed}(\Sigma, \Omega)$ has $\sigma[T].h \geq \Sigma.h_{\max} - 1$ (algorithm spec), combining with the previous inequality yields $h_c = \sigma[T].h = \Sigma.h_{\max} - 1$ and $\sigma[B^\star].h = \Sigma.h_{\max}$. The same height-advance argument applied to $\Sigma.J$'s own justification (its witnessing block has σ-height $\geq \Sigma.h_j + 1$, and that σ-height is $\leq \Sigma.h_{\max}$ by definition of $h_{\max}$) gives $\Sigma.h_j \leq \Sigma.h_{\max} - 1 = h_c$, while Lemma 11 applied to T_{lock}'s justification at h_c gives $h_c \leq \Sigma.h_j$. Hence $\Sigma.h_j = h_c$, and Lemma 13 forces $\Sigma.J = T_{\text{lock}}$.

$\Sigma.h_j = h_c = \Sigma.h_{\max} - 1$ gives $\Sigma.h_{\max} = \Sigma.h_j + 1$, satisfying the cascade gate, so getConfirmed walks from $\Sigma.J$. The return descends from $\Sigma.J = T_{\text{lock}}$ (existence: $B^\star \succeq \Sigma.J$ with $\sigma[B^\star].h = \Sigma.h_{\max}$ provides the required witness in $\mathcal{T}'(\Sigma)$ at σ-height $\Sigma.h_{\max}$, since any leaf $L \succeq B^\star$ inherits $\sigma[L].h = \Sigma.h_{\max}$ by state-height monotonicity). Hence $T \succeq \Sigma.J = T_{\text{lock}}$, with $\sigma[T_{\text{lock}}].h = h_c$ by Lemma 2 on the chain witnessing T_{lock}'s justification. □

Theorem 13 (Layer 1 leak fairness)**.** *Assume $f < n/3$ and all honest validators follow Definition 14. Let i be an honest validator, Σ its local store, Ω its auxiliary state, $T = \mathsf{getConfirmed}(\Sigma, \Omega)$ its canonical chain, and $h_c = \sigma[T].h$ the current canonical state-height. Let v be the vote i produces at the current slot. Let Σ' be any store obtained by processing additional blocks and votes on top of Σ, with auxiliary state Ω', such that $T' = \mathsf{getConfirmed}(\Sigma', \Omega')$ is an extension of T with $\Sigma'.\sigma[T'].h = h_c$, and suppose v is included on T'. Then for every slot processed on T' from that point onward, while canonical continues to extend T' and canonical state-height stays at h_c, the Layer 1 increment to $\sigma.\mathsf{penalties}[i]$ on canonical is zero.*

Proof. We show that processing v on T' sets $\mathsf{timeouts}[i] = \texttt{true}$ in $\sigma[T']$. By Section 1, $\mathsf{timeouts}[i]$ is set at processing time iff either (a) v is a timeout vote at the current state-height ($v.\mathsf{target} = \bot$ and $v.\mathsf{height} = \sigma[T'].h = h_c$), or (b) v's target is height-fresh on T' ($v.\mathsf{target} \preceq T'$ with $\sigma[v.\mathsf{target}].h = h_c$, equivalently $v.\mathsf{target}.\mathsf{slot} \geq \sigma[T'].s_h$ via Lemma 2).

Lemma 14 gives the dichotomy. In the timeout sub-case, $v = (i, h_c, \bot, \bot, \bot)$ with $\sigma[T'].h = h_c$ satisfies (a) directly. In the non-timeout sub-case, $v.\mathsf{target} \preceq T \preceq T'$ with $\sigma[v.\mathsf{target}].h = h_c$, so v is height-fresh on T' and (b) holds.

Either way, $\mathsf{timeouts}[i] = \texttt{true}$ in $\sigma[T']$. timeouts is per-state and resets only at a height advance; on any canonical chain extending T', σ at T''s descendants inherits $\mathsf{timeouts}[i] = \texttt{true}$ from $\sigma[T']$, and no height advance fires while canonical state-height stays at h_c, so $\mathsf{timeouts}[i] = \texttt{true}$ persists on canonical and Layer 1 does not bump i. □

5 Fork-choice specification

The store of Section 3 fixes a two-way cascade root and a viability filter, but leaves the choice among viable descendants of that root to an abstract auxiliary Ω (the argument of getConfirmed). This

section instantiates Ω concretely. Fork choice is organized in three layers, all consumed by a single procedure walk:

- **L1 — finality gadget (FG).** The Simplex-style height machine of Sections 1–3: heights advance by fixed-target justification or by timeout, $\Sigma.J/\Sigma.F$ are maintained with the lex-max rule and the F-filter, and the cascade root is $\Sigma.J$ when $\Sigma.h_{\max} = \Sigma.h_j + 1$ and $\Sigma.F$ otherwise (cascadeRoot, Section 5.2). E1 (Definition 9) remains the only slashing condition.
- **L2 — stabilization gadget (SG), latest messages.** The SG reads the head fields of all valid FG attestations received from the network, latest-per-validator with expiry (Section 5.1); block inclusion is neither required nor privileged. The SG fixes the walk's *anchor* in one of two ways (Section 5.2): a round's first-slot proposal may point to a block root; at voting time each validator accepts the root only if its previous-round direct support plus equivocation credit reaches $\frac{2}{3}$ of total stake. Absent an accepted pointed root, the anchor is the deepest block with latest-head-vote support $\geq \frac{2}{3}D$ reached by descent from the cascade root. There is a single grade-1 threshold, $\frac{2}{3}D$, at every height.
- **L3 — Goldfish (available chain).** The per-slot available-committee protocol, unchanged from its specification, which supplies the confirmation rule latest_confirmed_head (Section 5.3). Its duty is never gated by FG state.

The walk (Section 5.2) chains these: it fixes the anchor above (accepted fresh root, or the grade-1 descent as fallback), follows Goldfish from the anchor within the viable subtree, then applies the viability descent to restore the height bound at the seam. Section 5.5 specifies how an honest validator turns the walk output into an FG attestation (round-atomic construction, target rule, uniform confirmation and grade gates, and the empty vote), and Section 5.6 fixes the timing model and the named standing hypotheses used throughout Sections 5 and 6.

5.1 Latest head votes

Every FG attestation carries, in addition to the Section 1 vote fields, a *head field* $v.\mathsf{head} \in \text{Blocks}$: the block the signer's walk (Section 5.2) output at construction time. The head field is the signer's SG latest message. It does *not* enter the per-chain state transition of Section 1 (which reads only height, target, finalize_height, finalize_target); fork choice consumes it immediately after the attestation is received and validated.

Definition 15 (Live latest head vote)**.** Fix a validator's fork-choice store Σ at current slot s. Every valid FG attestation received over the network contributes its signed head field immediately; learning the same attestation from a block is merely another delivery path and creates no distinct object. For protocol expiry parameter $W > 0$, the attestation is *live* while $v.\mathrm{slot} + W > s$. For each validator i, its live latest head vote is the head field of its greatest-slot live attestation. A later-slot attestation supersedes the earlier one whether or not either is ever included in a block. A validator known to have equivocated contributes no live latest head vote; otherwise it contributes to at most one child subtree in each local view.

Definition 16 (Latest-head-vote weight)**.** Let $\mathcal{L}(\Sigma) \subseteq \{1, \ldots, n\}$ be the set of active, unslashed, non-equivocating validators holding a live latest head vote in Σ, and for $i \in \mathcal{L}(\Sigma)$ write head_i for i's latest head and w_i for its effective balance (stake weight). The *live latest-head-vote weight* is

$$D(\Sigma) \;\coloneqq\; \sum_{i \in \mathcal{L}(\Sigma)} w_i,$$

and, for a block c, its *latest-head-vote support* is the subtree-counted weight

$$S(c) \;\coloneqq\; \sum_{\{\, i \in \mathcal{L}(\Sigma) \,:\, c \preceq \mathsf{head}_i \,\}} w_i.$$

Known equivocators are excluded from both S (numerator) and D (denominator), as are inactive and slashed validators. We write D for $D(\Sigma)$ when Σ is clear.

S is monotone under $\preceq$: $c \preceq c'$ implies $S(c) \geq S(c')$, and $S(B_{\text{gen}}) = D$. Because a validator contributes to at most one child of any block (its live head lies in at most one child subtree), for a fixed block the head-vote supports of its children sum to at most D; hence, when $D > 0$, at most one child can carry $S \geq \frac{2}{3}D$, so the grade-1 descent below has a unique next step whenever it steps at all. The grades are not inclusion-defined: $\mathcal{L}$ and S are local network views. Post-GST delivery and the fixed freeze discipline relate honest views (Section 6.2); no proof waits for a proposer to include these votes.

5.2 Fresh-root syncing, anchors, and the walk

Definition 17 (Cascade root)**.** The *cascade root* of a store Σ is

$$\mathsf{cascadeRoot}(\Sigma) \;\coloneqq\; \begin{cases} \Sigma.J & \text{if } \Sigma.h_{\max} = \Sigma.h_j + 1, \\ \Sigma.F & \text{otherwise.} \end{cases}$$

This is exactly the walk-from block of getConfirmed (Figure 2, Section 3): $\Sigma.J$ when it sits at the frontier, and the always-viable $\Sigma.F$ (Lemma 7) when a timeout has advanced some chain past $\Sigma.J$'s height.

Fresh-root syncing

The walk's starting point below the cascade root is an *anchor*. The first proposer of a round points to a root; validators decide locally whether that root has an absolute quorum in the immediately preceding round. The proposal carries no votes or aggregate.

Definition 18 (Credited support and a fresh root)**.** Fix a validator u, a round r, and a block B. Let $V_u^r(B)$ be the validators from which u has received a valid round-r FG attestation whose head descends from B. Let E_u^r be the validators from which u has received two distinct valid round-r FG attestations. The *credited support* of B in u's view is

$$\mathsf{Credit}_u^r(B) \;\coloneqq\; w(V_u^r(B) \cup E_u^r)\,,$$

where each validator's effective balance is counted once. Thus a validator that u has seen equivocating is credited to B even if none of the copies seen by u supports B.

The block B is a *fresh root* in u's view for round $r+1$ iff $\mathsf{Credit}_u^r(B) \geq \frac{2}{3}n$, where n is the total active stake in the proposal's state. This is an absolute, view-independent denominator. "Fresh" refers only to use of the immediately preceding round; it is unrelated to the height freshness of an FG justification vote.

The equivocation term is deliberately generous to the proposer. If the proposer saw validator i vote for a descendant of B, while another validator saw two conflicting copies from i but not the proposer's copy, that validator still counts i for B. This is the second half of time-shifted quorum (TSQ): synchrony ensures that a vote used by an honest proposer is either visible to a validator by its voting action or that validator has evidence that the signer equivocated. Because the denominator is fixed and the local numerator only grows, no TSQ freeze is needed.

Lemma 15 (Fresh-root intersection)**.** *If honest validators u and v accept conflicting fresh roots B and B' for the same preceding round, then at least $n/3$ stake actually equivocated in that round. Hence, with less than $n/3$ equivocating stake, all accepted fresh roots of a round are compatible.*

Proof. Let $S = V_u^r(B) \cup E_u^r$ and $S' = V_v^r(B') \cup E_v^r$. Both have weight at least $2n/3$, so $w(S \cap S') \geq n/3$. Fix $i \in S \cap S'$. If i lies in either equivocation set, its two valid round-r attestations witness an actual equivocation. Otherwise one valid attestation from i supports B and one supports B'. Since the roots conflict, those attestations are distinct, so i again equivocated. Thus every validator in the intersection actually equivocated, irrespective of which copies either verifier received. □

Lemma 16 (Fresh-root synchronization)**.** *Fix consecutive rounds $r, r+1$ after* GST *under the standing assumptions and let an honest first-slot proposer of round $r+1$ point to a block B for which it saw direct round-r support of weight at least $2n/3$. At every honest validator's round-$r+1$ voting action, B is a fresh root in that validator's view. In particular, when $\beta < 1/3$, the highest common ancestor of the honest round-r heads has direct support greater than $2n/3$ and is available to an honest proposer.*

Proof. Consider each signer i in the proposer's direct-support set. The honest proposer re-gossips the supporting attestation. By the delivery-before-action discipline, every honest validator u has received that attestation by its voting action unless its relay path already stopped because u had retained two distinct round-r attestations from i; in that case $i \in E_u^r$. Consequently $V_p^r(B) \subseteq V_u^r(B) \cup E_u^r$, and u's credited numerator is at least the proposer's direct numerator, hence at least $2n/3$.

For existence, honest validators hold more than $2n/3$ stake and each emits one round-r attestation. Their highest common ancestor lies on every honest head, so every honest attestation directly supports it; after delivery an honest proposer sees all of that support. □

Pointing. A round's first-slot (r_0) proposer may put one block root B in its proposal. An honest proposer does so only after seeing at least $2n/3$ direct support for B in the preceding round; it does not use equivocation credit to construct the pointer, and it re-gossips the valid attestations it counted. The normal relay rule forwards a newly received valid round attestation unless the receiver has already retained enough conflicting data to mark that signer as a round equivocator. A validator gives the proposer the benefit of the doubt and accepts B exactly when its own credited support reaches $2n/3$ at the validator's voting action. Otherwise it ignores the pointer and uses the grade-1 fallback. For fixed B, each signer's contribution is monotone: receiving a supporting vote adds the signer, and receiving a conflicting copy can only turn the signer into equivocation credit. There is therefore no freeze and no carried certificate. Roots carry no cross-round state.

Remark 14 (Optional per-branch anchor monotonicity)*.* One may optionally harden pointing with a *chain-data* validity rule: a proposal's pointed anchor must extend the most recent pointed anchor among the proposal's own ancestors, so that the anchor sequence is monotone along every branch. This is a predicate on block content, checkable by any verifier, and introduces no validator state and no cross-round *store* field; it is *not* load-bearing for any theorem below and is listed only as optional hardening (Section 6.4).

The grade-1 anchor and the two grades

Absent an accepted pointed root, the anchor is fixed by a latest-head-vote descent under a single $\frac{2}{3}$ threshold. The two grade thresholds — the $\frac{2}{3}$ anchor and the $\frac{1}{3}$ conflict veto — are the only SG fork-choice objects, uniform at every height.

Definition 19 (Grade-1 anchor G_1)**.** The *grade-1 anchor* $G_1(\Sigma)$ is the latest-head-vote descent output under the $\frac{2}{3}$ threshold. If $D = 0$, return $\mathsf{cascadeRoot}(\Sigma)$ without descending. Otherwise start at $\mathsf{cascadeRoot}(\Sigma)$ and, while some child c in $\mathcal{T}'(\Sigma)$ has $S(c) \geq \frac{2}{3}D$, step into it (the unique such child, by the sum bound of Section 5.1); stop at the $\geq \frac{2}{3}$-crossed frontier. This is the walk's *fallback* anchor, used when the round's r_0 proposal does not point to an accepted fresh root (Definition 21).

Definition 20 (G_0-clearance)**.** A block T is *G_0-clear* in a store Σ iff no block B conflicting with T ($B \nsim T$) has latest-head-vote support $S(B) \geq \frac{1}{3}D$. Equivalently, every block with $S \geq \frac{1}{3}D$ is compatible with T. There is no unique "G_0 output": several blocks may simultaneously hold $S \geq \frac{1}{3}D$, and the veto form handles this directly. When $D = 0$, every block is G_0-clear by convention.

The walk

Definition 21 (The walk)**.** $\mathsf{walk}(\Sigma, \Omega)$ returns a block, computed in three phases (Figure 5).

1. *Anchor.* If the current round's r_0 proposal points to a block P that is a fresh root in the validator's local view at its voting action (Definition 18), and $\mathsf{cascadeRoot}(\Sigma) \preceq P$ with $P \in \mathcal{T}'(\Sigma)$, set the anchor $A \leftarrow P$. (If the pointed root is not a descendant of the cascade root — shallower than it, or on a conflicting branch — or if it has slipped behind the viability frontier, ignore it.) Otherwise set $A \leftarrow G_1(\Sigma)$, the grade-1 anchor (Definition 19). Either way $A \in \mathcal{T}'(\Sigma)$: the grade-1 anchor descends only into viable children from the always-viable cascade root, and the pointed root is adopted only when viable.

2. *Goldfish.* From the anchor A, follow the available chain *within the viable subtree*: $B \leftarrow \mathsf{goldfishHead}(A, \Omega)$ (Definition 23), the Goldfish walk anchored at A within Ω, stepping only into children in $\mathcal{T}'(\Sigma)$.

3. *Viability descent.* Apply Definition 22 — the phase-2 descent continued without its majority threshold — so the returned block sits at σ-height $\geq \Sigma.h_{\max} - 1$.

$\mathsf{walk}(\Sigma, \Omega)$ is the block after phase 3. All consumers — proposals, Goldfish votes, FG vote construction (Section 5.5), and confirmations — read the same walk.

Definition 22 (Viability descent)**.** Let B be the block after phase 2 of walk. The *viability descent* is the phase 2 Goldfish descent continued without its majority threshold: while $\sigma[B].h < \Sigma.h_{\max} - 1$, replace B by the viable child of B in $\mathcal{T}'(\Sigma)$ whose subtree carries the greatest previous-slot *participating* vote weight (subtree-counted, over the same vote set read by the phase 2 descent); ties, including the all-zero case, are broken deterministically. Halt as soon as $\sigma[B].h \geq \Sigma.h_{\max} - 1$. A viable child always exists until the bound is met, because a viable block has a viable leaf descendant (Definition 11) and any child lying under such a leaf is itself viable; the descent is *viability-gated* and resolves the one-height lag that Definition 11 tolerates. We do *not* claim the viable child is unique: below the frontier the viable subtree may branch — one child may carry its viable leaf at $\Sigma.h_{\max} - 1$ and another at $\Sigma.h_{\max}$, so both are viable — but the choice at such a seam is pinned by vote plurality over the same previous-slot vote set as phase 2, hence is a deterministic function of the view Ω rather than a free parameter. Thus all honest validators reading a common (merged) view make the same choice; only the postcondition of Lemma 17 — a viable block at σ-height $\geq \Sigma.h_{\max} - 1$ — is required downstream, and every viable branch meets it.

Figure 5: The walk

```
function cascadeRoot(Σ)
    return Σ.J if Σ.h_max = Σ.h_j + 1 else Σ.F
function walk(Σ, Ω)
    R ← cascadeRoot(Σ)
    if the current round's r_0 proposal points to P, Credit^{r-1}(P) ≥ (2/3)n, R ⪯ P, and P ∈ T'(Σ) then
        A ← P                                              ▷ locally accepted fresh root (Def. 18)
    else
        A ← G_1(Σ)                  ▷ fallback: latest-head-vote descent, ≥ (2/3)D from R (Def. 19)
    B ← goldfishHead(A, Ω)                          ▷ Goldfish from the anchor, within T'(Σ)
    while σ[B].h < Σ.h_max − 1 do   ▷ viability descent: phase-2 descent, majority gate dropped
        B ← viable child of B with maximal previous-slot participating vote weight, subtree-counted
                                                                          ▷ deterministic tiebreak
    return B
function getConfirmed(Σ, Ω)      ▷ instantiation of Figure 2: the walk output is the fork-choice head
    return walk(Σ, Ω)
```

The walk instantiates the abstract getConfirmed of Section 3: Ω is exactly the Goldfish available-chain state, and the descent among viable descendants of the cascade root is the anchor selection followed by the Goldfish-then-viability descent above. We keep two distinct notions: $\mathsf{getConfirmed}(\Sigma, \Omega) = \mathsf{walk}(\Sigma, \Omega)$ is the fork-choice *head* (the block votes and proposals build on), whereas the Goldfish *confirmed* head $\mathsf{latest\_confirmed\_head}(\Sigma, \Omega)$ (Definition 23) is in general an ancestor of it and is the object read by the confirmation gate (Definition 28) and by users. We verify that this concrete instantiation meets the postcondition on which all store theorems rest.

Lemma 17 (The walk meets the getConfirmed postcondition)**.** *For every store Σ and auxiliary Ω, walk(Σ, Ω) returns a block in $\mathcal{T}'(\Sigma)$ at σ-height $\geq \Sigma.h_{\max} - 1$; consequently getConfirmed instantiated by walk satisfies Corollary 1, and all store results of Section 3 (Theorems 7, 9, 10) carry over unchanged.*

Proof. The cascade root $\mathsf{cascadeRoot}(\Sigma)$ lies in $\mathcal{T}'(\Sigma)$: if the cascade gate holds then it is $\Sigma.J$ with $\sigma[\Sigma.J].h = \Sigma.h_j = \Sigma.h_{\max} - 1$ (Lemma 2), a viable leaf-witness by the argument of Corollary 1; otherwise it is $\Sigma.F$, viable by Lemma 7. Phase 1 fixes an anchor $A \in \mathcal{T}'(\Sigma)$: the grade-1 fallback $G_1(\Sigma)$ descends only into viable children from the (viable) cascade root and so stays in $\mathcal{T}'(\Sigma)$ (Definition 19), and the pointed anchor is adopted only when $P \in \mathcal{T}'(\Sigma)$ (Definition 21). Phase 2 then only ever steps from a block to a child that is itself in $\mathcal{T}'(\Sigma)$ (Goldfish descends the accepted tree and, restricted to $\mathcal{T}'(\Sigma)$, every block it can reach past A retains a viable leaf descendant — a block leaves $\mathcal{T}'$ only when it has no viable leaf above it, and then it has no children in $\mathcal{T}'$ to step into). Thus after phase 2, $B \in \mathcal{T}'(\Sigma)$, so B has a viable leaf descendant L with $\sigma[L].h \geq \Sigma.h_{\max} - 1$ (Definition 11). If already $\sigma[B].h \geq \Sigma.h_{\max} - 1$ the while-loop of phase 3 is skipped; otherwise each iteration steps into a viable child (one always exists while $\sigma[B].h < \Sigma.h_{\max} - 1$, since a viable block has a viable leaf descendant above the threshold), strictly increasing $\sigma[B].h$ (slots and hence state-height increase along $\preceq$, Lemma 3) until the bound is met. The loop terminates at a viable leaf at the latest, regardless of which viable child is chosen at any branching seam — the viability descent pins that choice by previous-slot vote plurality (Definition 22), but the termination argument does not depend on the pinning. Hence the return lies in $\mathcal{T}'(\Sigma)$ at σ-height $\geq \Sigma.h_{\max} - 1$, which is precisely the postcondition asserted (and proved abstractly) in Corollary 1; every downstream store theorem uses only that postcondition. □

5.3 Goldfish layer and confirmation

The available chain is the Goldfish protocol, taken as a black box and unchanged from its specification. We restate only its interface and its confirmation rule, both of which the walk consumes.

Definition 23 (Goldfish available chain and confirmation)**.** Each slot s has a pseudorandomly sampled *available committee* of 512 validators and one *available proposer*. At the slot start the proposer publishes its beacon block carrying the *merged view* (Definition 30), the merge taken over the view frozen at the previous slot's view-merge freeze; committee members then vote for a head at the attestation deadline. The merge covers *blocks and votes only*, exactly as in base Goldfish; latest FG head votes are ordinary network votes and need no additional carrier or inclusion path. The fresh-root pointer in a round's r_0 proposal (Definition 18) is only a root; validators test it against their live previous-round FG view at voting time, with equivocations credited to the root. The Goldfish walk $\mathsf{goldfishHead}(B, \Omega)$ from an anchor B descends by *previous-slot participant majority*: at each block it steps into the child whose subtree holds a majority of the previous slot's *participating* committee votes (subtree-counted), with same-slot proposals passing through.

Two confirmation rules read the committee attestations. *(a) Fast confirmation* outputs, immediately at $t + 2\Delta$, a block confirmed by a 75% *absolute* quorum of the slot's committee (a fraction of the full committee, not of the participants). It is included mainly for confirmation latency in the deployed protocol; the healing arguments of this document do *not* use it. *(b) Available confirmation* is the rule this document uses everywhere: $\mathsf{latest\_confirmed\_head}(\Sigma, \Omega)$ is the deepest block confirmed by the Goldfish rule under a 50% *relative* quorum of the participating committee, evaluated with one slot of delay through a *time-shifted quorum* — the participating vote set is *frozen* at slot n's $t + 2\Delta$ and the quorum is evaluated at slot $n + 1$'s $t + 2\Delta$ (Definition 30). The rule is *floorless*: it imposes no minimum participation and no FG-derived floor. Under $f < \frac{1}{2}$ of each participating committee it is safe and live within a slot.

Structural confirmation consistency. By the delivery-before-action discipline (Lemma 18), every honest attestation cast at its $t+\Delta$ deadline is delivered by $t+2\Delta$ and so lies in *every* honest validator's frozen set; honest frozen sets therefore differ only by adversarially-timed stragglers, and all honest validators evaluate the *same* 50% relative quorum at slot $n + 1$'s $t + 2\Delta$. The time shift thus turns confirmation consistency from a timing *assumption* into a property of the construction, with the residual straggler analysis deferred to the $\mathsf{latest\_confirmed\_head}$ interface item (Remark 29(2)). This time-shifted quorum is the *tip* confirmation rule's relative quorum — the sacrificial available layer, where relative counting is unavoidable — and is unrelated to the SG grade thresholds over latest FG head votes.

The available-vote duty and $\mathsf{latest\_confirmed\_head}$ are *never* gated by FG state (see the gate-scoping remark, Remark 16).

We keep two confirmation notions separate.

Definition 24 (Safe confirmation)**.** A validator *safe-confirms* a block B in its store Σ iff B is available-confirmed ($B \preceq \mathsf{latest\_confirmed\_head}(\Sigma, \Omega)$, Definition 23) *and* B is G_0-clear in its view (Definition 20) — no block conflicting with B has latest-head-vote support $\geq \frac{1}{3}D$. The *safe-confirmed head* C is the deepest safe-confirmed block — the deepest G_0-clear ancestor of $\mathsf{latest\_confirmed\_head}(\Sigma, \Omega)$ — and it is the confirmed head read by the FG gates (Section 5.5) and by the healing argument (Section 6). Safe confirmation is an *internal* protocol notion; the user-facing available confirmation (Definition 23) is untouched by it and is never gated by SG grades or FG state (Remark 16).

Because G_0-clearance is monotone along a chain — a block conflicting with B also conflicts with every ancestor of B — the safe-confirmed head is a well-defined ancestor of $\mathsf{latest\_confirmed\_head}(\Sigma, \Omega)$: if B is G_0-clear and $B' \preceq B$, then B' is G_0-clear.

5.4 Nonjustifiable heights

There is a single height class that changes the FG rule, and it exists only under sustained non-finality. It is a deterministic predicate on chain state and introduces no message, trigger, or protocol phase. Fix protocol parameters $K \geq 1$ (*nonjustifiable-height spacing*) and $D_{\text{debt}} \geq 1$ (*finality-debt threshold*).

Definition 25 (Nonjustifiable height)**.** On a chain with state σ, the system is in *finality debt* at height h when $h - \sigma.h_j^F > D_{\text{debt}}$, where $\sigma.h_j^F$ is the height of the most recently finalized block $\sigma.F$ (equivalently $h - \mathsf{height}(\Sigma.F) > D_{\text{debt}}$ in the store). A height h is *nonjustifiable* iff the system is in finality debt at h and $K \mid h$ (every K-th height under debt). At a nonjustifiable height the height-closing logic *never produces a justification*: the justified-checkpoint computation of processHeightEvents returns empty (no T is treated as justifiable), so height h can advance only by the timeout branch, and honest validators cast no target vote at h — only timeout votes, or the empty vote when gated out (Section 5.5). Every height that is not nonjustifiable is an ordinary *justifiable* height.

The grade thresholds and the FG gates are *uniform* at every height: there is no separate recovery-height class and no grade-activation predicate. The grade-1 anchor G_1 (Definition 19) uses the $\frac{2}{3}D$ threshold and G_0-clearance (Definition 20) the $\frac{1}{3}D$ conflict veto at all heights; the only effect of a nonjustifiable height is to disable the FG justification branch there, forcing the height to advance by timeout. A height may also end *justification-free in fact* — advancing by timeout with no justification firing — and such a height is then *sealed* (Lemma 19, condition (ii)); unlike a nonjustifiable height, this is a fact used inside proofs, never a class predicate. Under continued finality debt nonjustifiable heights recur every K heights (Definition 25), so the next such height is always at most K heights away.

5.5 FG vote construction

An honest validator turns the walk output into an FG attestation. The construction *refines the target selection of* getBaseVote (Definition 14) and then composes the *unchanged* getFinalVote on top of it, so the E1 layer (Lemma 12) and per-height justification uniqueness (Lemma 13) are preserved verbatim: what follows only fixes *which* target is fed to getFinalVote, and when it is replaced by a timeout vote or an empty vote (the latter carrying an empty voted checkpoint and only getFinalVote's finalize gate). The gates below are *uniform at every height* — there is no recovery-height class — and read the *safe-confirmed head* C (Definition 24) rather than the raw Goldfish-confirmed head.

Definition 26 (Round-atomic construction)**.** FG time is grouped into *rounds* of 32 consecutive slots (Section 5.6); r_0 denotes a round's first slot. For this document all FG/SG voting happens *at once* at r_0: all FG/SG vote *content* of a validator's round attestation — its walk output (head field), target, vote height, and finalize piggyback — is fixed at r_0's freeze from the validator's view at that instant, and the attestation is *emitted* at r_0. (Distributing emission over the round's later slots is a deployment option, deliberately not modeled here.) Each validator emits *one* FG attestation per round. A validator that has cast a (non-$\perp$ target) vote at a height never casts a *fresh* target at that height (*no-retarget*); this is exactly the lock/timeout adjustment of getFinalVote and is load-bearing for accountable safety (Lemma 12).

Definition 27 (Interval-first target rule and protected repeat)**.** Let $W = \mathsf{walk}(\Sigma, \Omega)$ be the round's walk output and $h_c = \sigma[W].h$ its state-height. The *target* is the *interval-first block*: the earliest block on W's chain with slot $\geq \sigma[W].s_h$, i.e. the block that opened height h_c on that chain (equivalently, the $\preceq$-least ancestor $T \preceq W$ with $\sigma[T].h = h_c$). The vote height is h_c. This T is the target passed to getBaseVote/getFinalVote in place of the raw getConfirmed block.

Protected repeat (imported from the source healing draft). An honest validator that has already cast a target vote at height h_c ($\Delta.T(h_c) \neq \perp$) *never casts a timeout vote at* h_c. At a repeat opportunity at h_c: if the current round's interval-first target still equals its saved target ($T = \Delta.T(h_c)$) and the target gate (Definition 28(a)) holds, it re-emits $(\Delta.T(h_c), h_c)$ — a re-emission of the *same* target, not a

fresh retarget, and hence E1-safe (Lemma 12); otherwise it casts the empty vote (Definition 29). Either way it does *not* set $\Delta.\tau(h_c)$, so a later finalize piggyback at h_c can still pass the $\Delta.\tau(h_c) = \texttt{false}$ gate of getFinalVote (Definition 14). When the validator is *locked* at h_c ($\Delta.\lambda(h_c) = \texttt{true}$) this is exactly the lock branch of getFinalVote, which re-submits $\Delta.T(h_c)$; the protected repeat extends the same no-timeout discipline to the not-yet-locked case, replacing the base spec's re-emission rule.

Definition 28 (Uniform confirmation and grade gate)**.** Let T be the interval-first target of the current round (Definition 27), at height h_c, and write C for the validator's *safe-confirmed head* (Definition 24). The gate reads two thresholds on C: the *caught-up* condition $\sigma[C].h \geq \Sigma.h_{\max} - 1$ (the safe-confirmed head has reached the frontier interval) and the stricter *into-the-interval* condition $\sigma[C].h \geq h_c$ (it has reached the very height being voted). The gate is the same at every height, with the single difference that a *nonjustifiable* height (Definition 25) admits no target vote.

- *Nonjustifiable height h_c.* The validator casts a timeout vote $(\perp, h_c)$ iff it is caught up ($\sigma[C].h \geq \Sigma.h_{\max} - 1$); otherwise it casts the empty vote (Definition 29). No target vote is ever cast at a nonjustifiable height.
- *Justifiable height h_c.* In order:
  (a) *target vote* (T, h_c) iff *both* $C \succeq T$ (the safe-confirmed head is T or a descendant) *and* T is G_0-clear in its view (Definition 20);
  (b) else *timeout vote* $(\perp, h_c)$ iff the safe-confirmed head is into the interval, $\sigma[C].h \geq h_c$ — the validator confirmed into the interval but its interval-first target is not a target case (either $C \not\succeq T$, or T fails G_0-clearance);
  (c) else the *empty vote* Checkpoint() (Definition 29): the safe-confirmed head has not reached the interval, $\sigma[C].h < h_c$.

  Subject to the *protected repeat* (Definition 27): if $\Delta.T(h_c) \neq \perp$, branch (b)'s timeout is replaced by re-emitting $\Delta.T(h_c)$ (when it is still the interval-first target and case (a) holds) or by the empty vote, never a timeout at an already-targeted height.

Every FG vote requires the safe-confirmed head to reach the voted interval, and a target vote requires in addition that the specific interval-first T be confirmed ($C \succeq T$) and conflict-free in latest head votes (G_0-clear). Case (a) implies case (b)'s condition, since $C \succeq T$ gives $\sigma[C].h \geq \sigma[T].h = h_c$; and because C is G_0-clear (Definition 24), $C \succeq T$ already forces T G_0-clear (any block conflicting with T conflicts with C), so (a)'s G_0 condition is automatic — we keep it stated for emphasis. *Timeout votes are confirmation-gated but not G_0-gated:* a validator confirmed into the interval times the height out even when its interval-first target is G_0-vetoed. This is deliberate and preserves liveness — once any interval block is safe-confirmed the frontier can always advance, and is never held hostage by a conflicting latest-head-vote configuration holding $\geq \frac{1}{3}D$; the empty vote, which advances nothing, fires only for a validator that has *not* confirmed into the interval. Whichever branch fires, the target T (when non-$\perp$) is passed through getFinalVote unchanged, so E1-safety holds (Lemma 12).

Definition 29 (Empty vote)**.** When the gate of Definition 28 admits neither a target nor a timeout vote — the caster's safe-confirmed head has not reached the voted interval, or the protected repeat blocks a timeout at an already-targeted height — the validator does *not* suppress its attestation. It emits the *empty vote* Checkpoint(): a vote (Definition 4; no new message type) with an *empty voted checkpoint*, target $= \perp$ *and* height $= \perp$ (the empty Checkpoint()), whose head field head is nonetheless *populated* with the walk output W (its SG latest message) and which carries its getFinalVote finalize piggyback independently. Its processVote effect (Figure 1) is:

field	empty-vote effect	rationale
$\mathsf{timeouts}[i]$	*unchanged* ($\mathsf{height} = \bot \neq \sigma.h$)	no timeout-cert contribution
$\mathsf{targets}[i]$	unchanged ($\mathsf{target} = \bot$)	no justification contribution
P	add i iff its finalize gate passes	finality piggyback, per $\mathsf{getFinalVote}$
$v.\mathsf{head}$	received as i's latest head vote	SG presence maintained

Both coordinates of the voted checkpoint are empty, so the empty vote is invisible to the height machine: it contributes to neither a justification quorum (Definition 6, which counts only non-$\bot$ targets) nor a timeout certificate (Definition 7, which counts only current-height timeout markers). It advances nothing. Only its head field enters the latest-message store (Definition 15) and its finalize fields the finality tally, so a validator keeps its SG head vote (and any finality piggyback) flowing while abstaining, without confirmation, from the height's FG decision. The empty vote replaces the base spec's whole-vote abstention: at a gated-out height an honest validator emits it *everywhere* (justifiable and nonjustifiable heights alike), so head fields — the fresh-quorum material — keep flowing at all heights. In particular the empty vote is neutral for the safe-confirmation certificate of Section 6 (Lemma 23): it can never help justify or time out a height. Unlike a timeout vote, however, it does *not* set $\mathsf{timeouts}[i]$ and so does *not* exempt i from the Layer 1 stall penalty (Figure 3, Section 4); this leak exposure now applies at *all* gated-out heights and is discussed in Remark 15. Its finalize fields follow the $\mathsf{getFinalVote}$ gate independently, so no P-entry is created that a plain finalize-bearing timeout vote would not also have created.

Remark 15 (The empty vote sets no marker; the certificate property and leak exposure). The empty vote must not set the timeout marker, and with an empty voted checkpoint (Definition 29) it sets none. Were it to set $\mathsf{timeouts}[i]$, a height could cross the timeout threshold on a quorum of gate-blocked empty votes, which witness no confirmation. With the empty vote excluded, a height advances by timeout only on a $2n/3$ quorum of *genuine*, confirmation-gated timeout or target votes (Definition 28), each of whose honest casters safe-confirmed into the voted interval. This is what lets reaching $h + 2$ from a justifiable height $h + 1$ certify a safe confirmation into $h + 1$'s interval on *every* advance path, with no justify-versus-timeout dichotomy (Lemma 23). Because the empty vote never sets a marker, and every marker-setter safe-confirmed into the interval, the certificate is a property of the construction rather than a fact about a particular advance schedule.

The price is a Layer 1 leak exposure. Because a gate-blocked honest validator no longer sets $\mathsf{timeouts}[i]$, the Layer 1 stall guard (Figure 3, Section 4) bumps it while it emits empty votes — in principle a fairness leak against an honest validator that the gate has silenced, now possible at any gated-out height. We argue it is negligible under the healing dynamics. Leak units accrue at *epoch* granularity (Section 4), whereas gate-blocked silence lasts only the sub-epoch interval between an honest-proposer round and the ensuing Goldfish confirmation (Theorem 26); sub-epoch silence is invisible to an epoch-granularity leak. Even a pathological transient in which the gate persists across a full epoch costs a single validator on the order of one epoch's leak increment — of order 10^{-7} of effective balance under the deployment leak parameters (Section 4, Theorem 12) — and does not compound while finality is not being denied. If exact Layer 1 neutrality is wanted, the listed alternative is a per-validator *presence field* that exempts the empty vote from the Layer 1 guard *without* counting it toward the timeout certificate (Remark 29, Section 6.4); it is leak-neutral without relying on the epoch-granularity argument, at the cost of one extra state field.

Figure 6: Uniform FG vote construction (refines getBaseVote of Figure 4)

```
1: function getFGVote(Σ, Ω, Δ)                          ▷ called once per round, content frozen at r_0
2:     W ← walk(Σ, Ω);    h_c ← σ[W].h
3:     T ← earliest block on W's chain with slot ≥ σ[W].s_h                ▷ interval-first, Def. 27
4:     C ← safe-confirmed head of (Σ, Ω)                ▷ deepest G_0-clear confirmed block, Def. 24
5:     h_f ← σ[W].h_j;    T_f ← σ[W].J                                    ▷ finalize piggyback
6:     if h_c is nonjustifiable then                                      ▷ no target ever, Def. 25
7:         if σ[C].h ≥ Σ.h_max − 1 and Δ.T(h_c) = ⊥ then    ▷ caught up; protected repeat, Def. 27
8:             v ← getFinalVote(Δ, (i, h_c, ⊥, h_f, T_f))                 ▷ timeout vote
9:         else
10:            v ← emptyVote(Δ, h_f, T_f)      ▷ empty vote (incl. at an already-targeted height),
   Def. 29
11:    else                                          ▷ justifiable height: uniform gate, Def. 28
12:        targetCase ← (C ⪰ T) and (T is G_0-clear)                      ▷ Def. 20
13:        if Δ.T(h_c) ≠ ⊥ and Δ.λ(h_c) = false then                      ▷ protected repeat, Def. 27
14:            if targetCase and T = Δ.T(h_c) then
15:                (h_f, T_f) ← finalizeGate(Δ, h_f, T_f);    v ← (i, h_c, Δ.T(h_c), h_f, T_f);
   rememberVote(Δ, v)                                                     ▷ re-emit
16:            else
17:                v ← emptyVote(Δ, h_f, T_f)      ▷ never a timeout at an already-targeted height
18:        else if targetCase then                                        ▷ (a) target vote (first vote / lock)
19:            v ← getFinalVote(Δ, (i, h_c, T, h_f, T_f))                 ▷ E1 layer, unchanged
20:        else if σ[C].h ≥ h_c then          ▷ (b) into the interval, G_0-vetoed / off-chain: time out
21:            v ← getFinalVote(Δ, (i, h_c, ⊥, h_f, T_f))     ▷ genuine timeout vote (sets timeouts[i])
22:        else                                                           ▷ (c) not into the interval
23:            v ← emptyVote(Δ, h_f, T_f)
24:    v.head ← W                                  ▷ head field always populated (SG latest message)
25:    return v
26: function emptyVote(Δ, h_f, T_f)      ▷ empty voted checkpoint + E1-safe finalize piggyback
27:    (h_f, T_f) ← finalizeGate(Δ, h_f, T_f)
28:    v ← (i, ⊥, ⊥, h_f, T_f)                  ▷ empty voted checkpoint ⇒ no timeout marker
29:    rememberVote(Δ, v);    return v
30: function finalizeGate(Δ, h_f, T_f)                    ▷ same finalize gate as getFinalVote
31:    if Δ.T(h_f) = T_f and Δ.τ(h_f) = false and T_f ≠ ⊥ then
32:        return (h_f, T_f)
33:    else
34:        return (⊥, ⊥)
```

Remark 16 (Gate scoping: gates never touch user confirmations). The G_0-clearance and confirmation gates of Definition 28 constrain *FG target votes* (and, through the safe-confirmed head, the "safe" labelling of confirmations that the healing argument uses, Section 6) *only*. They never gate the floorless user-facing available confirmation rule latest_confirmed_head (Definition 23). At low participation the G_0 veto silences FG target votes — heights stall, which is correct, since finality requires a $2n/3$ quorum in any case — while available-chain confirmations keep flowing at $f < \frac{1}{2}$ of each participating committee, unchanged. Gating the user-facing confirmation on SG grades would let an adversary holding $\geq \frac{1}{3}D$ veto available-chain liveness, violating the $f < \frac{1}{2}$ guarantee; the scoping above avoids this.

5.6 Timing model and standing assumptions

We fix one timing model for the whole document: an available-chain slot grid, over which FG rounds are built, with FG attestation content frozen at each round's first slot. The liveness argument of Section 6 references only round-level quantities.

Definition 30 (Slots, rounds, and voting time)**.** Physical time is partitioned into discrete *slots* indexed by $k \in \mathbb{N}$, with start time $\mathsf{start}(k)$ and $\mathsf{start}(k+1) = \mathsf{start}(k)+4\Delta$ for the network delay bound Δ: a slot spans 4Δ, with *phase deadlines* at Δ intervals from the slot start $t \coloneqq \mathsf{start}(k)$. Writing $\mathsf{vote}(k) \coloneqq t+\Delta$ for the slot's *attestation (voting) time*, the phases are:

- t — *beacon block.* The slot's proposer publishes its *beacon block*: a light object carrying the merged view (Definition 23), the merge taken over the view frozen at the previous slot's view-merge freeze (below). Under ePBS the execution *payload* is revealed by the builder on a parallel track at $\approx t+\Delta$; payload propagation — the heavy object — overlaps the later phases and costs no additional phase.
- $t+\Delta$ — *attestation deadline.* The available committee casts its attestations on the beacon block.
- $t+2\Delta$ — *evaluation point.* Four things happen at once: (i) *PTC votes* are due, cast having seen the payload during $[t+\Delta,\, t+2\Delta]$; (ii) *fast confirmation* — an immediate confirmation by a 75% absolute quorum of the slot's committee attestations (Definition 23); (iii) *available confirmation of the previous slot* — evaluate the 50% relative quorum over the vote set *frozen* at the previous slot's $t+2\Delta$ (the time-shifted quorum, Definition 23); and (iv) *freeze* the vote set that the *next* slot's available-confirmation evaluation will use.
- $t+3\Delta$ — *view-merge freeze.* The view, both attestations and PTC votes, is frozen for the next slot's proposer to merge; the view-merge window is $[t+3\Delta,\, t+4\Delta]$.
- $t+4\Delta = \mathsf{start}(k+1)$*:* the next slot begins.

Three observations. The schedule is 4Δ *even with ePBS* — the same as Goldfish-with-fast-confirmation without ePBS — because payload propagation parallelizes with the attestation and PTC phases and the first Δ is only beacon-block propagation. Fast confirmation is obtained *for free*: it sits between the attestation deadline and the freeze in any case. The one genuine cost of the structure is the final view-merge Δ.

Slots carry the available-chain (Goldfish) committees and proposers of Definition 23. FG time is grouped into *rounds* of 32 consecutive slots; r_0 denotes the first slot of a round and freeze its *freeze instant* — the view-merge freeze at r_0's $t+3\Delta$. Each honest validator invokes the uniform FG construction (Figure 6) once per round, fixing its attestation content at r_0's freeze from the merged view it then holds, and produces *at most one* FG attestation per round (Definition 26).

Lemma 18 (Delivery before action)**.** *Fix the slot schedule of Definition 30 and work after* GST*, so message delay is at most Δ. Every object an honest validator publishes at its phase deadline — a beacon block at the slot start t, an attestation (carrying its SG latest head vote) at $t+\Delta$, a PTC vote at $t+2\Delta$ — is received by every honest validator by the next phase deadline, hence before any honest action that reads it (each such action sits at least Δ after the publication it consumes). Consequently the* honest-published *component of cross-view latest-message disagreement at any honest action point is zero: at any freeze, every honest head vote published at a strictly earlier phase deadline is common to all honest views; only adversary-published objects, whose publication time the schedule does not constrain, can be delivered to some honest validators before a freeze and to others after it.*

Proof. The phases of Definition 30 are spaced by exactly Δ and delay is $\leq \Delta$, so each publication clears before the next deadline — one line per phase:

published	delivered by	first consuming action
beacon block (t)	$t+\Delta$	attestation deadline ($t+\Delta$)
attestation ($t+\Delta$)	$t+2\Delta$	$t+2\Delta$ evaluations / electorate freeze
PTC vote ($t+2\Delta$)	$t+3\Delta$	view-merge freeze ($t+3\Delta$)

Each consuming action lies at or after the "delivered by" time, so the object is in hand; and any freeze of a *later* slot lies a further multiple of Δ on. Thus every honestly-published head vote is common to all honest views at that action and at every later freeze, leaving zero honest-published contribution to the cross-view disagreement; only adversary-published objects, whose publication time the schedule does not constrain, can reach some honest validators before a freeze and others after it. □

Definition 31 (Time-dependent dishonest count)**.** For each time t, f^t denotes the number of validators that are dishonest at time t. We assume f^t is monotonically non-decreasing in t: once a validator is dishonest, it remains so. We write β for the corresponding fraction of total stake, $\beta := f^t/n$ at the time of reference.

Definition 32 (Standing assumptions for round r)**.** A round r (with first slot r_0) satisfies the *standing assumptions* when (a) $\mathsf{start}(r_0) - \Delta \geq \mathrm{GST}$ (synchrony), (b) more than half of the validators online at r_0's freeze are honest (*online-honest majority*, per participating available committee), (c) every honest validator is online at r_0's freeze (*all-honest-awake*), and (d) every FG attestation cast by an honest validator in round r is gossiped to every honest validator within Δ and is eventually received by an honest available-chain proposer that includes it before the end of round r (*honest-relay*). The gossip clause drives SG grades; the inclusion clause is used only to close FG certificates.

Fixed proposer sequence. The proposer sequence is *fixed and exogenous*: this document does not model RANDAO or seed manipulation, so the identity of each slot's proposer — in particular each round's first-slot proposer at r_0 — is given in advance and independent of chain content. Proposer-sequence manipulation (grinding of first-slot positions, seed biasing) is therefore deliberately out of scope (Section 6.4); the honest-proposer waits of the liveness leg (Theorem 26) are read off this fixed sequence.

For an honest validator v and time t, write Σ_v^t, Ω_v^t for v's store and auxiliary state at t, and $C_v^t := \mathsf{latest\_confirmed\_head}(\Sigma_v^t, \Omega_v^t)$ for its Goldfish-confirmed head (distinct from the fork-choice head $\mathsf{getConfirmed}(\Sigma_v^t, \Omega_v^t) = \mathsf{walk}(\Sigma_v^t, \Omega_v^t)$, of which it is in general an ancestor).

Named hypotheses. The timing and threshold hypotheses used by reference throughout Sections 5 and 6 — the synchrony gap and slot-phase spacing (with confirmation consistency structural via the time-shifted quorum, Definition 23), the FG supermajority ($\beta < 1/3$), the Goldfish committee-majority bound ($f < 1/2$ per participating committee), and data commonality, which together underwrite the single-step graded-delivery bound of Section 6.2 — are collected once, as the standing hypotheses of the healing argument (Assumptions 18–22, Section 6.3), and are used by reference from here on.

6 Healing from asynchrony

After a period of asynchrony the finality gadget may be stuck: heights have advanced by timeout, no recent justification anchors the frontier, and the adversary may selectively deliver equivocal latest messages once the network heals. This section shows that the graded fork choice of Section 5 lets finality *heal* after a stall. Accountable safety (Theorem 2) is never at risk — it holds unconditionally at $< n/3$ slashable and is not re-proved here. What must be shown is that finality *resumes*: that the frontier advances and the safe-confirmed head converges to a common, canonical block. That is the participation-conditional content, and it rests on one small unconditional fact.

The one unconditional healing-specific fact is the *safe-confirmation certificate* (Lemma 23): because the empty vote sets no timeout marker, every advance of a height to $h+2$ witnesses that some honest

validator safe-confirmed a block in $h+1$'s interval. It invokes no proposer sequence, no timing, no participation, and no grade-delivery assumption — only the gate, the empty-vote marker semantics, and the threshold $2/3 - \beta > 1/3$. FG non-interference during $h+1$ — no height-h justification can re-enter the fork choice — is the equally unconditional seal (Lemma 19), also cited, not restated. The *healing theorem* (Theorem 23) turns that per-caster certificate into a *common*, *canonical* safe confirmation with no conflicting honest anchor, and re-establishes finality by induction. This is *conditional* on at least 2/3 of the stake being honest and online, and it is closed once honest latest messages have crossed one post-GST delivery boundary: it consumes the graded delivery of Section 6.2, anchor intersection, and latest-message concentration. Inclusion is needed later to register FG quorums, but no longer to stabilize the grades. The *liveness leg* (Theorem 26) carries the timing conditions and shows the frontier advances, supplying the healing theorem's hypotheses — in particular the delivery/freeze event; healing then completes (Theorem 28).

6.1 Regeneration

Recall from Section 5.4 that a *nonjustifiable height* (Definition 25) is one at which, under finality debt, the state transition's justified-checkpoint computation returns empty, so honest validators cast no target votes there. A height may also end justification-free *in fact* — advancing by timeout with no justification firing. Either way the height is *sealed*, and its seal is the structural fact on which FG non-interference in the healing argument rests: it removes any height-H justification from the fork choice throughout the tenure of the next height.

Definition 33 (Sealed height)**.** A height H is *sealed* for a node if no justification at height H can ever fire on any chain the node processes; equivalently, no block ever attains a $2n/3$ justification quorum (Definition 6) at height H.

Lemma 19 (Regeneration)**.** *Suppose height H is sealed (Definition 33). Then throughout the tenure of height $H+1$ — every store state with $\Sigma.h_{\max} = H+1$ — the cascade gate $\Sigma.h_{\max} = \Sigma.h_j + 1$ fails, so* getConfirmed *walks from $\Sigma.F$ (Figure 2); in particular no height-H justification can ever be the fork-choice root. Two sufficient conditions for H to be sealed:*

(i) by construction — *H is nonjustifiable (Definition 25): the justification branch is disabled at H and honest validators cast no target votes there, so no block collects a $2n/3$ quorum;*

(ii) in fact — *the honest target votes at H are* sub-completable at all times*: at no time does any block receive height-H target votes from a $2/3 - \beta$ fraction of the total stake, so no block ever reaches $2n/3$ even with all $\leq \beta n$ adversarial target votes. (Honest target votes at H are pinned by no-retarget (Definition 26), so this is a well-defined all-time hypothesis on the final honest tally, not a snapshot.)*

Under continued finality debt nonjustifiable heights recur every K heights (Definition 25), so the next one sealed by construction *is always at most K heights away. The* seal *does* not *propagate under a bare timeout advance: a height may advance by timeout while remaining completable. With the empty vote out of the timeout certificate (Definition 29), the $2n/3$ timeout markers come only from confirmation-gated votes — genuine timeout votes and the height-fresh target votes, which also set the marker (Definition 7). If some honest validators cast a target vote $(T, H+1)$ that does not by itself reach $2n/3$ while the height times out on the combined markers, then T still carries $\geq (2/3 - \beta)n$ honest weight, and the adversary may later reveal its $\leq \beta n$ withheld target votes on a chain still at height $H+1$ to complete $J(H+1) = T$. That case is not a seal but a pending justification of a safe-confirmed block: T was safe-confirmed by its honest casters (the gate cast $(T, H+1)$ only with $C \succeq T$, Definition 28(a)), so by per-height justification uniqueness (Lemma 13) any $J(H+1)$ that later fires is of that block, which heals rather than threatens safety (Theorem 23).*

Proof. Gate failure. Any chain that reaches state-height $H+1$ did so by a height advance from H (Lemma 1). Since no justification at H can fire, that advance is a timeout cert (Definition 7), which

leaves h_j unchanged (Lemma 3); so on that chain the last justified height is at most $H-1$. More generally, while $\Sigma.h_{\max} = H+1$, a justification at height h' forces some chain to state-height $h'+1 \le \Sigma.h_{\max} = H+1$, hence $h' \le H$; as $h' = H$ is excluded by the seal, every fired justification has height at most $H-1$, so $\Sigma.h_j \le H-1$. Thus $\Sigma.h_{\max} = H+1 \ge \Sigma.h_j + 2$, the cascade gate $\Sigma.h_{\max} = \Sigma.h_j + 1$ fails, and getConfirmed walks from $\Sigma.F$ (Figure 2). Because a height-H justification would carry key $(H, \cdot)$ and the gate selects $\Sigma.J$ only when $\Sigma.h_j = \Sigma.h_{\max} - 1 = H$, and that never occurs during the tenure, no height-H justification is ever the walk-from block.

(i) Nonjustifiable. At a nonjustifiable height the justified-checkpoint computation returns empty (Definition 25), so the justification branch of processHeightEvents never fires at H; and honest validators cast no target votes at H, so any block's height-H target support is at most $\beta n < n/3 < 2n/3$. No $2n/3$ quorum can form, so H is sealed.

(ii) Sub-completable. Suppose at no time does any block T have height-H target votes from a $2/3-\beta$ fraction of the total. A justification quorum for T needs $2n/3$ target votes for T; adversarial validators supply at most βn, so honest validators would have to supply at least $(2/3-\beta)n$ — contradiction with the all-time hypothesis. No block is ever justifiable at H, so H is sealed.

Cadence and non-propagation. A nonjustifiable height H is sealed by construction (condition (i)); under continued finality debt such heights recur every K heights (Definition 25), so the next guaranteed seal is at most K heights away. Whether $H+1$ is in turn sealed is settled by conditions (i)–(ii) applied to $H+1$. A bare timeout advance does *not* seal: the empty vote no longer sets the timeout marker (Definition 29), but the height-fresh target votes do (Definition 7), so $H+1$ can cross the $2n/3$ timeout threshold on confirmation-gated timeout and target votes while some target T still holds $\ge (2/3-\beta)n$ honest votes that did not by themselves reach justification; then T remains completable, and a later reveal of the adversary's $\le \beta n$ withheld target votes can fire $J(H+1) = T$ on a chain still at height $H+1$. That is a pending justification of a safe-confirmed block, handled by the healing theorem (Theorem 23), not by the seal. □

Remark 17 (The seal is not a protocol phase). Sealedness is a predicate on chain state, not a protocol phase: condition (i) is the deterministic nonjustifiable-height predicate (Definition 25), and condition (ii) is decided by the height's own target tally. No message, trigger, or ceremony is introduced.

6.2 Graded delivery

The grades are computed over live latest head votes — the head field $v.\mathsf{head}$ of every valid FG attestation received from the network, latest-per-validator, in-window W, with known equivocators excluded (Definition 15). Write $S_u(B)$ for the latest-head-vote support of block B and D_u for the total live latest-head-vote weight in validator u's view (Definition 16). Recall (Definition 19) that the grade-1 anchor $G_1 = B$ means $S(B) \ge \frac{2}{3}D$, and (Definition 20) that a target T is G_0-*clear* iff no block conflicting with T has latest-head-vote support $\ge \frac{1}{3}D$.

The healing theorem needs one cross-view fact about grade-1 anchors: a block that one honest validator sees with $\ge \frac{2}{3}$ support is seen by every honest validator with $\ge \frac{1}{3}$ support at the next action boundary. We first note that the walk's *fallback* grade-1 anchor is such a block in its holder's own view, then prove the cross-view step with per-view denominators. A *pointed* root is *not* covered by this step — its credited previous-round support is a different tally from the live latest-message grade — and is protected separately by fresh-root intersection (Lemma 15), as Corollary 2 makes precise.

Lemma 20 (Fallback anchor grade-1 support)**.** *Let $A = G_1(\Sigma_u)$ be the* fallback grade-1 anchor *an honest validator u adopts in a round when its round's r_0 proposal does not point to a locally accepted fresh root (Definition 21), evaluated at u's round freeze after* GST*. Then either $A = \mathsf{cascadeRoot}(\Sigma_u)$, or $S_u(A) \ge \frac{2}{3}D_u$ in u's own view. A* pointed *root $A = P$ carries* no *such latest-head-vote-support guarantee: $S_u(P) \ge \frac{2}{3}D_u$ need not hold.*

Proof. When the pointed branch is not taken, $A = G_1(\Sigma_u)$: the grade-1 descent of Definition 19 either does not step, leaving $A = \mathsf{cascadeRoot}(\Sigma_u)$, or steps into a child with $S_u \ge \frac{2}{3}D_u$, the last such child

being A. The pointed case carries no latest-head-vote-support guarantee: its local acceptance reads the credited previous-round tally (Definition 18), which neither carries votes in the proposal nor writes to the receiver's latest-message store (Definition 15). Hence no lower bound on $S_u(P)$ follows — before the honest head votes have concentrated, P may have support well below $\frac{2}{3}D_u$, or below $\frac{1}{3}D_u$. Pointed anchors are therefore protected not by local grade support but by fresh-root intersection (Lemma 15); see Corollary 2. □

Lemma 21 (Single-step graded delivery)**.** *Assume $\beta < 1/3$ (Assumption 21), data commonality (Assumption 22), and delivery before action (Lemma 18). Assume the honest validators are online and have published a live head vote, so their common live weight is at least a $(1-\beta)$ fraction of total stake. For any block X and any two honest validators u, v evaluating grades at the same action boundary after the Δ-delivery boundary following those honest publications (or once head votes have concentrated, Lemma 22): if $S_u(X) \geq \frac{2}{3}D_u$ in u's view, then $S_v(X) \geq \frac{1}{3}D_v$ in v's view. Because the live-honest-vote hypothesis is participation-dependent, this lemma and its corollary (Corollary 2) belong to the dependency set of the conditional healing theorem (Theorem 23), not of the unconditional safe-confirmation certificate (Lemma 23).*

Proof. By delivery before action (Lemma 18), every live honest head vote published before the preceding phase boundary is common to all honest views at the action boundary. Write w_H for their common weight and H_X for their common weight supporting X. The remaining, view-dependent latest messages are adversarially published; write a_u, a_v for their live weight present in the two views, and $a_u^X \leq a_u$ for the part supporting X in u. The adversary holds at most a β fraction of the stake while the common honest live messages carry at least a $1-\beta$ fraction, so the per-view adversary weight is capped by

$$a_u,\, a_v \;\leq\; \rho\, w_H, \qquad \rho \coloneqq \frac{\beta}{1-\beta} \;<\; \tfrac{1}{2} \quad (\text{since } \beta < \tfrac{1}{3}).$$

From $S_u(X) = H_X + a_u^X \geq \frac{2}{3}D_u = \frac{2}{3}(w_H + a_u)$ and $a_u^X \leq a_u$,

$$H_X \;\geq\; \tfrac{2}{3}(w_H + a_u) - a_u \;=\; \tfrac{2}{3}w_H - \tfrac{1}{3}a_u \;\geq\; \tfrac{2}{3}w_H - \tfrac{1}{3}\rho\, w_H \;=\; \tfrac{2-\rho}{3}\, w_H.$$

In v's view $S_v(X) \geq H_X$ and $D_v = w_H + a_v \leq (1+\rho)w_H$, so

$$S_v(X) - \tfrac{1}{3}D_v \;\geq\; \tfrac{2-\rho}{3}\, w_H - \tfrac{1+\rho}{3}\, w_H \;=\; \tfrac{1-2\rho}{3}\, w_H \;>\; 0,$$

using $\rho < \frac{1}{2}$. Hence $S_v(X) \geq \frac{1}{3}D_v$. □

Corollary 2 (No honest anchor conflicts with a safe confirmation)**.** *Suppose an honest validator v safe-confirms a block C (Definition 24) and every honest anchor descends from a common cascade root $R \preceq C$. Assume moreover the standing hypotheses and that honest head votes lie on C's chain, so that the honest round-r attestations give C direct fresh support greater than $2n/3$ (Lemma 16 once concentration has placed honest heads on C's chain, Lemma 22). Then no honest validator's adopted anchor conflicts with C. The two anchor kinds are handled by two routes, both* conditional *on participation: the fallback grade-1 anchor by graded delivery, the pointed root by fresh-root intersection.*

Proof. Let A be an honest u's adopted anchor (Definition 21).

Fallback grade-1 anchor $A = G_1(\Sigma_u)$. By Lemma 20, either $A = \mathsf{cascadeRoot}(\Sigma_u) = R \preceq C$, which does not conflict with C, or $S_u(A) \geq \frac{2}{3}D_u$. In the latter case, if A conflicted with C, single-step graded delivery (Lemma 21, using its live-honest-vote hypothesis) would give $S_v(A) \geq \frac{1}{3}D_v$ in v's view — a block conflicting with C holding $\geq \frac{1}{3}D_v$ head-vote support, contradicting the G_0-clearance of C in v's safe confirmation (Definition 20).

Pointed anchor $A = P$. Here P has at least $2n/3$ credited previous-round support in u's view but need not carry grade-1 support (Lemma 20), so the graded-delivery route does not apply. Instead use C itself: the honest round-r attestations give it direct support greater than $2n/3$, so it is a fresh root in every honest view after delivery. If P conflicted with C, these would be two conflicting accepted

fresh roots, forcing at least $n/3$ of the stake to have equivocated (Lemma 15) — impossible under the standing assumption.

Either way A does not conflict with C. The pointed-anchor route consumes the participation-dependent premise that C has greater than $2n/3$ direct fresh support (honest heads on C's chain); this is why the corollary, like Lemma 21, is a dependency of the conditional healing theorem (Theorem 23) rather than of the unconditional safe-confirmation certificate (Lemma 23). □

The liveness leg needs the complementary fact that, once honest head votes concentrate, no conflicting branch can hold $\geq \frac{1}{3}D$. This is a consequence of latest-message supersession, not of any move-counting.

Lemma 22 (Latest-message concentration)**.** *After* GST*, suppose the honest validators are online and publish their next head votes on the current chain. Within Δ, latest-per-validator supersession (Definition 15) places every honest validator's live latest head vote on that chain in every honest view. Every conflicting branch then has support below $\frac{1}{3}D$, and this persists while honest head votes remain on the chain. No block inclusion is required. A stale head vote from a validator that remains offline expires after W slots.*

Proof. The new honest messages supersede the same validators' older messages as soon as they are received; delivery before action makes them common within Δ. Fix an honest view and write w_H for its common honest live weight and a for its adversarial live weight. Any conflicting branch has support at most a, while $D = w_H + a$. The standing hypotheses give $w_H \geq 1 - \beta$ and $a \leq \beta$ as fractions of total stake, hence $2a < w_H$ for $\beta < 1/3$. Therefore $a < \frac{1}{3}(w_H + a) = \frac{1}{3}D$. Further honest messages remain on the current chain and can only supersede same-validator stale votes; an offline validator's unsuperseded message ceases to be live after W slots. □

6.3 The healing theorem

Standing hypotheses. The two legs draw on the following named hypotheses, collected once here as the canonical timing and threshold assumptions of the whole document and used by reference throughout; the timing model of Section 5.6 points here rather than restating them. Their quantification is revisited in Section 6.4.

Assumption 18 (Network delay and phase spacing). After GST the message delay is bounded by Δ, and each slot phase spans Δ (Definition 30), so an object published at one phase deadline is in every honest view by the next. Confirmation consistency is then *structural*, not a separate timing hypothesis: the available confirmation is evaluated over a vote set frozen one phase before the view-merge freeze (the time-shifted quorum, Definition 23), so a block an honest validator confirms is seen by every honest validator before the freeze at which it fixes its vote content.

Assumption 19 (Honest supermajority). Honest validators hold at least 2/3 of the total active stake.

Assumption 20 (Goldfish committees). In each per-slot available committee the adversarial fraction is $f < 1/2$ of the participating members.

Assumption 21 (FG adversary bound). The adversarial FG stake is $\beta < 1/3$ of the total.

Assumption 22 (Data and vote commonality). After GST, honest blocks and valid honest FG attestations propagate within Δ. Thus any two honest validators' block sets differ by at most one Δ-window of in-flight blocks, and every honest head vote published before the preceding phase boundary is present in both latest-message stores. No inclusion or backfill premise is imposed on grade inputs.

The uniform confirmation-and-G_0 gate of Section 5.5 makes every honest target vote a vote for a safe-confirmed block: by Definition 28(a) an honest validator casts (T, h) only when $C \succeq T$ for its safe-confirmed head C, so T is safe-confirmed in the caster's view (Definition 24); otherwise it casts a confirmation-gated timeout vote (if it confirmed into the interval) or the empty vote (if it did not, Definition 29).

Lemma 23 (Safe-confirmation certificate)**.** *Assume fewer than $n/3$ of the stake is slashable and $\beta < 1/3$ (Assumption 21, the adversary bound — not a participation hypothesis). On any advance of a justifiable height $h+1$ to $h+2$, some honest validator safe-confirmed a block in $h+1$'s interval. This is the unconditional fact that the empty-vote-sets-no-marker design buys (Remark 15); its proof uses no cross-view graded delivery, fresh-quorum relation, latest-message concentration, or honest participation — only the gate, the empty-vote marker semantics, and the threshold arithmetic $2/3 - \beta > 1/3$.*

Proof. An advance of $h+1$ is a $2n/3$ quorum of votes setting the timeout marker (Definition 7). The empty vote sets no marker (Definition 29), so it is never in the quorum; every marker is a target vote (case (a) of Definition 28) or a timeout vote (case (b)), and more than $(2/3 - \beta)n > n/3$ are honest (Assumption 21). Each such honest caster had its safe-confirmed head C_i into the interval at its freeze, $\sigma[C_i].h \geq h+1$: case (a) requires $C_i \succeq T$ so $\sigma[C_i].h \geq \sigma[T].h = h+1$, and case (b) requires $\sigma[C_i].h \geq h+1$ outright. So each such honest caster safe-confirmed a block in $h+1$'s interval; taking any one gives the claim. The argument reads only the gate (Definition 28), the empty-vote marker semantics (Definition 29), and $2/3-\beta > 1/3$; it consumes no cross-view graded delivery, anchor relation, or participation. □

The certificate is unconditional but it is only a *per-caster* witness — *some* honest validator safe-confirmed *some* block in the interval. Turning it into a *common*, *canonical* safe confirmation that resumes finality is what healing adds, and it is conditional on honest participation. Accountable safety (Theorem 2) is untouched throughout and is not re-proved here; per-height justification uniqueness (Lemma 13) and the seal / cascade gate (Lemma 19) — the FG non-interference facts — are likewise cited unconditionally, not restated.

Theorem 23 (Finality heals)**.** *Assume fewer than $n/3$ of the stake is slashable and, as the* healing hypothesis*, that* at least $2/3$ of the total stake is honest and online — *equivalently $\beta < 1/3$ (Assumption 21) with full honest participation (Assumption 19 together with the all-honest-online clause of the standing assumptions, Definition 32) — with post-*GST *operation and the slot schedule (Assumptions 18, 22) and $f < 1/2$ per participating available committee (Assumption 20). Suppose height h finalizes a block B, and let C be a deepest block safe-confirmed by an honest validator in $h+1$'s interval on an advance to $h+2$ (Lemma 23). Then, once honest validators have published their next head votes on the safe-confirmed chain and one post-*GST *Δ-delivery boundary has passed (Lemmas 22 and 21):*

(1) (Common, canonical safe confirmation.) *C is common to all honest validators, no honest validator's adopted anchor conflicts with C (Corollary 2), and with $f < 1/2$ per committee C is and remains canonical for every honest validator. Every honest fork-choice root during $h+1$'s tenure descends from B (Lock-in, Theorem 9), so B lies on every honest anchor and $B \preceq C$.*

(2) (Finality resumes; induction.) *Every honest canonical chain is at state-height $\geq h+1$ and contains C; any $J(h+1)$ that fires is the unique justification (Lemma 13) of a safe-confirmed block on C's chain; latest-head-vote support concentrates on C's chain (Lemma 22), so no future anchor conflicts with C. Finality re-establishes at $h+1$: the finalize piggyback finalizes the justified block on the next advance, and the protected repeat (Definition 27) keeps honest finalize gates open, so the hypotheses recur at height $h+1$.*

Accountable safety is untouched independently of the healing hypothesis: no block conflicting with B is ever finalized (Theorem 2), at $< n/3$ slashable and with no timing or participation assumption.

Proof. The proof is a chain of gates and impossibilities; no case tree over adversarial schedules appears. Throughout, the safe-confirmation certificate (Lemma 23) is the unconditional starting point, and the healing hypothesis enters only to make the certified confirmation common and canonical, closed by network delivery of the next honest head votes.

(1) The fork-choice root descends from B. For any honest node that has processed a block B' with $\sigma[B'].J = B$ and $\sigma[B'].h_j = h$, Lock-in (Theorem 9) makes every getConfirmed return a descendant

of B at all future times, so its walk output, anchor, and cascade root descend from B. A node still lagging — at $\Sigma.h_{\max} = h+1$ with $\Sigma.h_j \le h-1$, so $\mathsf{cascadeRoot}(\Sigma) = \Sigma.F \preceq B$ (Definition 17) — has not yet locked onto B; but under the standing hypotheses viability together with latest-message concentration (Lemma 22) bring its walk onto B's chain within a round. (The nonjustifiable corollary routes exactly this step through $\mathsf{cascadeRoot} = \Sigma.F$ unconditionally, and is the cleaner template; Corollary 3.) Fix the deepest block R finalized in every honest view; $R \preceq \Sigma.F \preceq$ every honest cascade root and every honest anchor (Lemma 5), and $R \preceq B$.

(1) A deepest common safe confirmation. By the safe-confirmation certificate (Lemma 23) some honest validator safe-confirmed a block in $h{+}1$'s interval. All honest safe-confirmed blocks are available-confirmed, hence pairwise compatible by Goldfish safety ($f < 1/2$, Assumption 20); they lie on one chain, and we take C to be a deepest one, safe-confirmed by some honest v with G_0-clearance in v's view (Definition 24). Then $R \preceq B \preceq C$ (C is in $h+1$'s interval, above B), so every honest anchor descends from the common root $R \preceq C$. Under the standing hypotheses honest available votes concentrate on C's chain (Goldfish majority, $f < 1/2$), so an honest bloc of head votes lands on C's chain and the honest round-r attestations give C direct fresh support greater than $2n/3$ (Lemmas 16, 22). Corollary 2 then applies by both its routes — fallback grade-1 anchors by graded delivery, pointed roots by fresh-root intersection — and no honest anchor conflicts with C. (This is the single cross-view inference of the argument, read at the round freeze; its full freeze-alignment formalization — that v's G_0-evaluation and each anchor-holder's evaluation are related by Lemma 21 at the next action boundary after honest head-vote delivery — is Open Item 1, Remark 29.)

(1) Canonical and permanent. C is Goldfish-confirmed by an honest validator; with $f < 1/2$ per committee the Goldfish majority reaches C and honest available votes keep it canonical (Definition 23), so C is common to all honest validators. Once honest head votes on C's chain have propagated, latest-message concentration (Lemma 22) leaves every branch conflicting with C below $\frac{1}{3}D$ in every honest view, so no future grade-1 anchor G_1 conflicts with C; and an accepted fresh root conflicting with C would require at least $n/3$ equivocating stake (Lemma 15). Hence C remains canonical.

(2) No reorg and induction. By (1) every honest canonical chain is at state-height $\ge h+1$ (any block at the frontier descends from B, and the height filter never reverts below it) and contains C (by (1), C is canonical). Any $J(h+1)$ that fires has a target T' that its honest contributors safe-confirmed (the gate cast $(T', h+1)$ only with $C_i \succeq T'$, Definition 28(a)); T' is compatible with C (both safe-confirmed, hence available-confirmed and compatible by Goldfish safety), so T' is on C's chain, and per-height uniqueness (Lemma 13) makes it the unique justifiable block at $h+1$. No conflicting safe confirmation and no conflicting $J(h+1)$ can form. Finally, finality re-establishes at $h+1$: once T' is justified at $h+1$, honest validators that cast $(T', h+1)$ carry the finalize piggyback $(\mathsf{finalize\_height}, \mathsf{finalize\_target}) = (h+1, T')$, and the protected repeat (Definition 27) keeps $\Delta.\tau(h+1) = \texttt{false}$ so their finalize gate stays open (Definition 14); when $2n/3$ such finalize-bearing votes accumulate, T' finalizes at $h+1$. The hypotheses of the theorem then hold at height $h+1$ with $B \leftarrow T'$, and the argument recurs. □

Corollary 3 (Nonjustifiable heights heal)**.** *Suppose height H is nonjustifiable (Definition 25), hence sealed (Lemma 19(i)). Then the seal supplies, with no prior finalization to bootstrap, both FG non-interference at H (in place of a height-H finalization) and the healing theorem's common-root premise at $H+1$: no $J(H)$ exists, and throughout $H+1$'s tenure the cascade gate fails, so every honest fork-choice root is that node's own $\Sigma.F$ and no height-H justification is ever the root (Lemma 19). Write B for the* deepest block finalized in every honest view *— the nodes' own $\Sigma.F$ may differ but all descend from B (Lemma 5). Every honest node has finalized B, so Lock-in (Theorem 9) makes every honest anchor descend from B. The per-caster certificate (Lemma 23) then holds at $H+1$, and with B below $H+1$'s interval the healing theorem (Theorem 23) applies: reaching $H+2$ certifies a common safe confirmation C in $H+1$'s interval, no honest anchor conflicts with C, and finalizing at $H+1$ heals. Since nonjustifiable heights recur every K heights under finality debt (Definition 25), a fully-sealed healing attempt occurs on that cadence, requiring no prior finalization to bootstrap.*

Proof. Sealedness of H is Lemma 19(i); its conclusion (cascade gate fails throughout $H+1$'s tenure;

getConfirmed walks from $\Sigma.F$; no height-H justification is ever the root) is FG non-interference at H, the seal standing in for a height-H finalization. Because every honest node has finalized the common block B, Lock-in (Theorem 9) applied with $F \coloneqq B$ gives $B \preceq \Sigma.F \preceq$ every honest cascade root and anchor, so the common root $R = B$ of the healing theorem's step (1) is available *without* the lagging-node concentration argument — this is the cleaner, fully-sealed template. The per-caster certificate (Lemma 23) and both steps of the healing theorem (Theorem 23) then apply verbatim at $H + 1$. The recurrence of nonjustifiable heights is Definition 25. □

Remark 24 (What is already unconditional, and what healing adds). *Accountable safety is unconditional and already established.* No block conflicting with a finalized B is ever finalized (Theorem 2), at $< n/3$ slashable, with no timing, participation, or honesty-online hypothesis. Healing neither touches this nor re-proves it; the healing argument only cites it. The other unconditional facts it leans on are equally pre-existing: FG non-interference during $h + 1$ is the seal (Lemma 19), and per-height justification uniqueness is Lemma 13.

The one *healing-specific* unconditional fact is the safe-confirmation certificate (Lemma 23): reaching $h + 2$ witnesses that *some* honest validator safe-confirmed a block in $h + 1$'s interval. It invokes no honest-proposer round, no slot deadline, no participation, and no graded-delivery premise — only the uniform gate (Definition 28), the empty-vote marker semantics (Definition 29), and the threshold arithmetic $2/3 - \beta > 1/3$. But it is a *per-caster witness*, not canonicality.

What *healing adds* — that the certified C is *common* to all honest validators, that *no honest anchor conflicts* with it, that it *stays canonical*, and hence that finality *resumes* — is *conditional* on at least $2/3$ of the stake being honest and online (Theorem 23), and is closed after honest head votes cross one post-GST delivery boundary. Commonality comes from Goldfish canonicality ($f < 1/2$), no-anchor-conflict from graded delivery (fallback anchors) and anchor intersection (pointed anchors, Corollary 2), and permanence from latest-message concentration (Lemma 22) — or W-window turnover for offline validators. We therefore do *not* bill canonical convergence or no-anchor-conflict as pure state-and-threshold, and do not claim canonicality before the delivery-aligned action boundary.

Remark 25 (Timeouts are confirmation-gated, not G_0-gated). The per-caster certificate (Lemma 23) rests on a deliberate scoping choice in the gate (Definition 28): a timeout vote requires only that the caster have safe-confirmed into the interval, *not* an additional G_0 condition beyond the one already folded into safe confirmation. Gating timeouts more strongly would stall the frontier exactly when some conflicting branch holds $\geq \frac{1}{3}D$, contradicting the requirement that timeouts flow once any interval block is safe-confirmed. We therefore recover the safety of a pure-timeout advance not from a target's cast-time clearance but from the common safe confirmation C together with latest-message concentration (Lemma 22) — the same forward-in-time machinery that underwrites permanence in the healing theorem (Theorem 23). This keeps the user-facing confirmation rule ungated (Remark 16): the added condition is on the *vote*, not on latest_confirmed_head.

Theorem 26 (Healing — liveness leg)**.** *Assume the standing hypotheses 18–22, honest participation of at least* $2/3$ *of the total, and the fixed proposer sequence of Section 5.6, with future honest first-slot proposers and future honest ordinary proposers. Assume the ordinary Simplex frontier-liveness property under those conditions: at every post-*GST *current height, the height-fresh target votes and current-height timeout votes processed on the canonical chain eventually set timeout markers for at least* $2/3$ *of the total stake. This is the base protocol's general liveness obligation, not an additional healing or nonjustifiable-height premise. Starting after* GST *at a nonjustifiable height* H *(Definition 25), the frontier eventually makes both required advances,*

$$H \longrightarrow H + 1 \longrightarrow H + 2.$$

The first is a timeout advance (the justification branch is disabled at H*); the second is an ordinary justification or timeout advance at height* $H + 1$*. The second advance carries the safe-confirmation certificate (Lemma 23), and by then the honest head votes have become common by network delivery. Thus the healing theorem (Theorem 23, via Corollary 3) applies.*

Each common-anchor opportunity waits for an honest first-slot proposer (geometric expectation $1/(1-\beta)$ rounds under an independent typical sequence); each FG height transition separately waits for an honest proposer to include the ordinary attestations forming its certificate. Network delivery closes the grade input within Δ and adds no inclusion wait of its own. This theorem claims eventual liveness under those proposer-fairness conditions, not that both height transitions fit inside one first-slot-proposer wait.

Proof. Fix either current height $K \in \{H, H+1\}$. Gated-out honest validators cast the empty vote (Definition 29) rather than abstaining, so their head fields keep flowing and remain fresh-quorum material. When a round's first slot r_0 has an honest proposer, it points to the highest root with at least $2n/3$ direct support in the previous round (Lemma 16). By the voting action, every honest validator sees each supporting vote or an equivocation by its signer, so its credited numerator is no lower than the proposer's and it accepts the same root (Definition 18). Above the common anchor, the per-slot Goldfish vote view-merge makes the walk common at honest-proposer slots. Available votes concentrate on the common chain and the time-shifted quorum available-confirms an interval block one slot later (Definition 23). Once honest validators publish their next head votes there, network delivery supersedes their stale messages within Δ, putting every conflicting branch below $\frac{1}{3}D$ (Lemma 22); that interval block is therefore safe-confirmed (Definition 24). No block inclusion or W-window turnover is needed for online honest validators' concentration; W only bounds an offline validator's stale message.

At $K = H$, the height is nonjustifiable, so the target branch is disabled. The marker accounting does not discard validators that already voted at H: a processed height-fresh target vote set $\mathsf{timeouts}[i]$ when it was processed, and a later empty vote neither clears nor replaces that bit. In particular, votes for the unique lock target supplied by justification uniqueness (Lemma 13) remain part of the marker quorum on the canonical chain containing that lock. Once the common safe-confirmed head reaches H's interval, the remaining honest validators admitted by the gate emit current-height timeout votes (Definition 28). Ordinary Simplex frontier liveness and future honest-proposer inclusion therefore supply the 2/3 marker quorum. Since justification is disabled at H, the timeout branch advances the frontier to $H+1$.

Repeat the same convergence argument at $K = H+1$. The walk reaches a block at state-height $H+1$, and the common safe-confirmed head C reaches the interval-first target T. Latest-message concentration makes T G_0-clear, so the target gate opens for fresh validators. Existing height-fresh target votes already contribute markers; newly cast target and timeout votes contribute markers when a future honest proposer includes them. Ordinary Simplex frontier liveness supplies a 2/3 marker quorum. If one target also has 2/3, the justification branch advances first; otherwise the timeout branch advances. Either path reaches $H+2$.

The second advance has a $2n/3$ timeout-marker quorum and excludes empty votes, so Lemma 23 supplies its per-caster safe-confirmation certificate. Corollary 3 and Theorem 23 upgrade it to a common, canonical safe confirmation. The grade steps used only network delivery; the two inclusion waits above are exactly the remaining FG liveness obligation. After convergence, eventual justification and finalization are supplied by the same ordinary Simplex liveness argument, resuming finality on the common chain; nonjustifiable heights introduce no separate signing-history release requirement. □

Remark 27 (Below 2/3 participation). When participation drops below 2/3 of the total, no quorum of any kind forms and no height advances; the system runs on available-chain confirmations only, which continue to flow at $f < 1/2$ because the G_0-veto gates FG target votes and the safe-confirmation label *only*, never the floorless user-facing confirmation rule (Remark 16). Stabilization beyond Goldfish requires 2/3 support in any case — an accepted design decision (Section 6.4). The inactivity leak (Section 4) then grinds FG-silent stake: its guards read only FG participation fields (Figure 3), so a validator denying finality is pivotal every epoch, and the leak drives restoration over days to weeks (Theorem 12). This is the status quo under non-finality; it introduces no new mechanism.

Theorem 28 (Healing)**.** *Assume the standing hypotheses 18–22 and honest participation of at least* 2/3 *of the total, together with the ordinary Simplex frontier-liveness property of Theorem 26. After*

GST, *starting from a nonjustifiable height H (Definition 25), the protocol heals: the frontier reaches height $H+2$ (Theorem 26), and every justification that can ever fire at $H+1$ is a justification of a common safe-confirmed block that is and stays canonical on every honest chain (Theorem 23 via Corollary 3). The regime recurs on the nonjustifiable-height cadence (Lemma 19): if $H+1$ ends sub-completable it is sealed* in fact *and no justification can ever fire there, and the next guaranteed-sealed attempt is the next nonjustifiable height, at most K heights on; if instead $H+1$ times out while a completable safe-confirmed target remains, that target's justification — whenever it fires — is safe: accountable safety (Theorem 2) is untouched and per-height uniqueness (Lemma 13) pins it to the safe-confirmed block, as re-established at $H+1$ by the healing induction (Theorem 23(2)). Either way no honest chain is ever reorganized. The assumed ordinary frontier-liveness property subsequently supplies justification and finalization on that common chain.*

Proof. Immediate from the safe-confirmation certificate (Lemma 23), the healing theorem (Theorem 23), the liveness leg (Theorem 26), Corollary 3, and the cadence clause of Lemma 19. □

6.4 Open items

The following points remain to be pinned down.

Remark 29 (Open items).

(1) *Base-protocol frontier liveness.* The healing liveness leg invokes the ordinary Simplex argument that, under synchrony, 2/3 online participation, and honest-proposer inclusion, current-height markers eventually reach 2/3 on one canonical chain. Both height-fresh target votes and timeout votes set the marker. A later empty vote does not erase a previously set bit; in particular, votes from validators committed to the unique lock target still count on the canonical chain containing that lock. Therefore nonjustifiable heights require no special vote-history release or alignment premise. What remains to cite or prove separately is the protocol's general frontier-liveness theorem, including vote inclusion on the canonical chain.

(2) *The single-step freshness discipline.* The whole cross-view inference of the healing theorem is a single step: anchor adoption, grade evaluation, and safe confirmation are all read at the same round freeze, and Lemma 21 relates one honest evaluation to another at that freeze. What remains is a bookkeeping audit that every latest-head-vote-consuming action point (each vote construction and freeze evaluation) sits at least Δ after the publication of the messages it reads, so that delivery before action (Lemma 18) applies, together with a full formalization of the time-shifted quorum's residual straggler analysis (Definition 23): adversary-published objects whose delivery is split across a freeze are the only cross-view residual, and the per-view adversarial-message cap that absorbs them (Lemma 21) should be tied precisely to the schedule.

(3) *Behavior for $\beta \in (1/3, 1/2)$.* Above $\beta = 1/3$ honest stake alone cannot give a root $\frac{2}{3}$-absolute direct fresh support, and the single-step graded-delivery guarantee no longer holds ($\beta/(1-\beta) \geq 1/2$). A local grade-1 anchor may still form, but it no longer has the cross-view grade-0 relation required by healing. The protocol then runs on the available chain, F/J, and the inactivity leak alone. This is an accepted design decision: stabilization beyond Goldfish requires 2/3 support in any case (Remark 27), and finality-restoration in this regime is left to the leak (Theorem 12).

(4) *Pointer equivocation by the r_0 proposer.* A first-slot proposer that points to two conflicting roots in conflicting proposals is equivocating on its proposal — the existing proposal-equivocation handling applies, and the fault is confined to a single round. Each honest validator evaluates the root of whichever proposal it processes against its own credited previous-round support. Fresh-root intersection (Lemma 15) guarantees that two roots cannot both be accepted and conflict without at least $n/3$ equivocating stake, so healing safety is unaffected.

(5) *Layer-1 exposure of the empty vote.* The empty vote does not set $\mathsf{timeouts}[i]$ (Definition 29), so a gate-blocked honest validator is exposed to the Layer-1 stall leak while it emits empty votes, now at *any* gated-out height (Remark 15). The exposure is negligible under the healing dynamics — gate-blocked silence is sub-epoch while the leak accrues per epoch — but exact neutrality is not forced by the mechanism alone. The listed leak-neutral alternative is a per-validator *presence field* that exempts the empty vote from the Layer-1 guard without counting it toward the timeout certificate; it adds one state field. A precise bound for the sub-epoch exposure should be stated.

(6) *The optional per-branch anchor-monotonicity rule.* Pointing carries no cross-round validator state. The optional hardening of Remark 14 — a proposal's pointed anchor must extend the most recent pointed anchor among its own ancestors — is a chain-data validity predicate, not load-bearing for any theorem here; its interaction with liveness (does it ever reject an honest proposal?) and its exact encoding are left open.

(7) *The* $\mathsf{latest\_confirmed\_head}$ *interface to FG and SG.* latest_confirmed_head is a recently added, still underexplored part of the spec. The priority question is its *interface* to safe confirmation (Definition 24): the times at which the available-confirmation rule is run and the way its result is stored must be compatible with the freeze at which the gates read the safe-confirmed head C (Definition 28). Under the phase schedule (Definition 30) confirmation consistency is structural via the time-shifted quorum (Definition 23); the residual is that Δ bounds real delay within each phase (Assumption 18). The interface to *user* confirmation is important but secondary here.

(8) *RANDAO branch-relativity of committees.* Available committees are branch-relative through RANDAO; identity-keyed vote stores (equivocation keyed on validator, slot, and data, not on a per-position seat) are a spec-side prerequisite for the Goldfish layer to attribute votes correctly across branch families. The fixed-and-exogenous proposer sequence of Section 5.6 is a modeling choice and does not discharge this prerequisite. Proposer-sequence manipulation (RANDAO grinding, seed biasing of first-slot positions) is correspondingly out of scope.